

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме:

**ВЕБ И МОБИЛЬНОЕ ПРИЛОЖЕНИЕ ДЛЯ КОРПОРАТИВНОГО
ОБУЧЕНИЯ С ПОДДЕРЖКОЙ АДАПТИВНОГО ТЕСТИРОВАНИЯ**

по направлению 09.03.03 Прикладная информатика

Образовательная программа (профиль) «Корпоративные информационные
системы»

Студент: _____ / Мурадян Лусинэ Гамлетовна, 221–361/
подпись *ФИО, группа*

Руководитель ВКР: _____ / Гонтовой Сергей Викторович, доцент, к.н./
подпись *ФИО, уч. звание и степень*

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.03 Прикладная информатика
Образовательная программа (профиль) «Корпоративные информационные
системы»

Тема ВКР	Веб и мобильное приложение для корпоративного обучения с поддержкой адаптивного тестирования
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Клиент-серверная система корпоративного обучения с поддержкой адаптивного тестирования для персонализации учебного процесса сотрудников и студентов, а также предоставления организациям удобных средств управления курсами и аналитики результатов обучения.
Основные функции	1. Управление пользователями и ролями (обучающийся, преподаватель, администратор организации, системный администратор). 2. Создание и редактирование курсов, уроков и тестов преподавателями. 3. Назначение курсов пользователям и группам администраторами. 4. Прохождение курсов и тестов обучающимися с мобильного приложения. 5. Адаптивное тестирование с подстройкой сложности заданий по результатам. 6. Формирование рекомендаций по повторению материалов. 7. Аналитика и отчёты по прогрессу для администраторов и преподавателей. 8. Поддержка нескольких организаций (multi tenant архитектура).
Используемые технологии и платформы	Сервер: Python (Django REST Framework / FastAPI), PostgreSQL Мобильное приложение: Flutter (Dart) Веб-панель: React (JavaScript/TypeScript)

	Протоколы: REST-API, HTTPS Развёртывание: облачная инфраструктура
ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Анализ предметной области корпоративного электронного обучения и существующих LMS. 2. Анализ подходов к адаптивному тестированию и персонализации обучения. 3. Проектирование функциональности системы и ролей пользователей. 4. Проектирование архитектуры клиент серверной системы и базы данных. 5. Разработка модуля адаптивного тестирования и рекомендаций. 6. Проектирование и реализация мобильного приложения обучающегося. 7. Разработка веб панели администратора и преподавателя. 8. Реализация серверной части с REST API. 9. Тестирование системы и анализ информационной безопасности.
Состав технической документации	1. Техническое задание на разработку системы. 2. Программа и методика испытаний. 3. Руководство пользователя (мобильное приложение). 4. Руководство администратора (веб-панель). 5. Описание серверной части и API. 6. Спецификация требований к системе.
Состав графической части	1. Список графической части

ПЛАН РАБОТЫ НАД ВКР

[illegible]

РУКОВОДИТЕЛЬ ОП:

«__»____2026, _____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«__»____2026, _____ / Гонтовой Сергей Викторович, доцент, к.н. /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«__»____2026, _____ / Мурадян Лусинэ Гамлетовна, 221–361/ /
подпись *ФИО, группа*

АННОТАЦИЯ

Наименование работы: выпускная квалификационная работа на тему «Веб и мобильное приложение для корпоративного обучения с поддержкой адаптивного тестирования».

Цель работы — разработать и описать клиент–серверный программный комплекс корпоративного обучения с адаптивным тестированием для персонализации учебного процесса и поддержки нескольких организаций в одном развёртывании. Способы достижения: анализ предметной области и аналогов на основе формальных критериев и экспертных оценок; формирование требований и сценариев использования; проектирование архитектуры модульного монолита, инфологической и даталогической моделей базы данных; реализация серверной части на FastAPI, веб-панели на React и TypeScript, мобильного клиента на Flutter; описание алгоритмов адаптивного подбора вопросов, формирования рекомендаций и изоляции данных арендаторов; подготовка материалов внедрения, опытной эксплуатации, экономического обоснования и приложений по ГОСТ ЕСПД.[1, 2, 3, 4]

Практический результат — программный комплекс с REST API (/api/v1), мультиарендной моделью данных, ролями пользователей, адаптивным тестированием со шкалой сложности 1–5 и рекомендациями по «слабым темам». Новизна связана с явной документацией дискретного алгоритма адаптации сложности и связкой аналитики с повторным обучением в корпоративном контексте.[5, 6]

Работа состоит из введения, трёх глав, заключения, списка использованных источников и пяти приложений (техническое задание; программа и методика испытаний; опросные листы экспертов; руководства пользователя и администратора; листинги ключевого кода). Общий объём основного текста (без титульного листа, задания, аннотации и приложений) соответствует методическим рекомендациям; иллюстративный материал включает бо-

лее 15 рисунков и более 10 таблиц; библиографический список содержит 51 источник (нормативные документы, монографии, статьи, техническая документация), в том числе не менее трёх иностранных изданий; доля электронных ресурсов ограничена требованиями нормоконтроля. Результаты могут быть апробированы в виде демонстрации на студенческой конференции или пилотного стенда кафедры —

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	10
1 ПРЕДМЕТНАЯ ОБЛАСТЬ И ТЕХНОЛОГИИ	15
1.1 Предметная область корпоративного обучения и исходное состояние	15
1.2 Постановка проблемы и назначение системы	17
1.3 Требования к системе	20
1.4 Анализ аналогов	28
1.5 Выбор технологий и платформ	30
1.6 Выводы по главе	36
2 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ	37
2.1 Архитектура системы	37
2.2 Сценарии использования	39
2.3 Модель данных	42
2.4 Проектирование интерфейсов	45
2.5 Реализация web- и mobile-клиентов	48
2.6 Реализация серверной логики и алгоритмов	52
2.7 Выводы по главе	57
3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ ПРОГРАММНОГО КОМПЛЕКСА	58
3.1 Развёртывание программного комплекса	58
3.2 Опытная эксплуатация и тестирование	61
3.3 Анализ информационной безопасности	63
3.4 Экономическое обоснование разработки	64
3.5 Маркетинговая стратегия и план развития продукта	66
3.6 Выводы по главе	67
ЗАКЛЮЧЕНИЕ	68
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	70
ПРИЛОЖЕНИЕ А. ТЕХНИЧЕСКОЕ ЗАДАНИЕ	75

ПРИЛОЖЕНИЕ Б. ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ	80
ПРИЛОЖЕНИЕ В. ОПРОСНЫЕ ЛИСТЫ ЭКСПЕРТОВ	83
ПРИЛОЖЕНИЕ Г. РУКОВОДСТВА ПОЛЬЗОВАТЕЛЯ И АДМИНИСТРАТОРА	86
ПРИЛОЖЕНИЕ Д. ЛИСТИНГИ КЛЮЧЕВОГО КОДА	89

ВВЕДЕНИЕ

Выпускная квалификационная работа посвящена разработке корпоративной системы управления обучением, предназначенной для организации учебного контента, назначения курсов, контроля результатов и проведения адаптивного тестирования в рамках единой клиент–серверной платформы.[1]

Актуальность темы обусловлена тем, что в корпоративной среде обучение сотрудников всё чаще строится как непрерывный процесс, включающий адаптацию новых сотрудников (онбординг), обязательные программы, проверку знаний и повторение проблемных тем. При использовании разрозненных инструментов усложняется управление пользователями и ролями, затрудняется назначение курсов, отсутствует единый контур аналитики и слабо поддерживается персонализация учебной траектории. Дополнительным фактором является распространение мобильного обучения, при котором слушатели проходят курсы короткими сессиями со смартфона и ожидают простой навигации, быстрого доступа к материалам и прозрачной обратной связи по результатам тестирования.[7, 8]

Для корпоративных сценариев также существенна возможность обслуживания нескольких организаций в рамках одной платформы с логическим разделением данных, ролей и учебного контента. Такой подход позволяет использовать единое программное решение для нескольких арендаторов, сохраняя изоляцию данных и независимость администрирования.[9, 10]

Целевой аудиторией разрабатываемой системы являются системный администратор платформы, администратор организации, преподаватель и слушатель. Для административных ролей важны средства настройки организаций, пользователей, курсов и аналитики, а для слушателя — удобное прохождение назначенного обучения с мобильного устройства, получение результата тестирования и рекомендаций по повторению материала.

Цель работы — разработать корпоративную LMS с веб-панелью администрирования и мобильным клиентом слушателя, обеспечивающую управление учебным контентом, назначениями, адаптивным тестированием и аналитикой результатов обучения.

Для достижения поставленной цели необходимо решить следующие задачи:

- а) проанализировать предметную область корпоративного обучения и исходное состояние процессов;
- б) выявить основные проблемы организации учебного процесса и обосновать необходимость автоматизации;
- в) сформулировать функциональные и нефункциональные требования к системе;
- г) выполнить сравнительный анализ существующих LMS-решений и определить критерии сопоставления;
- д) обосновать выбор технологий и платформ для разработки программного комплекса;
- е) спроектировать архитектуру системы, модель данных и пользовательские сценарии;
- ж) реализовать серверную, веб- и мобильную части системы;
- з) описать алгоритмы адаптивного тестирования, формирования рекомендаций и изоляции данных арендаторов;
- и) сформировать сценарий тестирования требований по критическому пути пользователей и сценарий демонстрации текущей версии проекта.

Объектом исследования являются процессы организации корпоративного обучения, контроля прохождения курсов и оценки результатов слушателей в цифровой образовательной среде.

Предметом исследования являются методы и средства проектирования и разработки клиент–серверной информационной системы корпоративного

обучения, включающей веб-интерфейс администрирования, мобильный клиент слушателя, адаптивное тестирование и механизмы аналитики.

Методы исследования: системный анализ процессов корпоративного обучения; сравнительный анализ программных аналогов по формализованным критериям и весам; объектно-ориентированное проектирование архитектуры и базы данных; документирование требований и трассировка к тест-кейсам; экспертная оценка LMS по опросным листам (приложение В).[11, 12, 13]

Информационная база исследования: нормативные документы по оформлению отчётной документации и программной документации;[2, 14, 15, 16, 17, 18, 3, 4, 19] методические рекомендации вуза по ВКР;[1] монографии и учебники по проектированию информационных систем, базам данных и инженерии программного обеспечения;[20, 21, 22, 23] работы по теории тестирования и адаптивному тестированию;[24, 5, 6, 25] официальная техническая документация используемых платформ.[26, 27, 28, 29, 30]

Гипотеза исследования состоит в том, что для корпоративного сегмента средней численности достаточно связки модульного монолита на стороне сервера с отдельными клиентами и логической изоляцией данных арендаторов при хранении в общей СУБД; эмпирическая проверка выполняется через опытную эксплуатацию стенда и прогон тест-кейсов приложения Б.

Практическим результатом работы является программный комплекс, реализованный в виде монорепозитория и включающий серверное приложение на основе FastAPI, веб-панель администратора и преподавателя на React и TypeScript, а также мобильное приложение слушателя на Flutter. Серверная часть обеспечивает аутентификацию, управление организациями, пользователями, курсами, уроками, тестами, рекомендациями и аналитикой. В системе реализованы разграничение доступа по ролям, логическая мультиарендная изоляция данных (с поддержкой нескольких арендаторов на одном

экземпляре БД), адаптивный подбор вопросов в тестах и формирование рекомендаций по слабым темам.

Практическая значимость полученного результата заключается в том, что он демонстрирует возможность организации корпоративного обучения в едином цифровом контуре: от подготовки контента и назначения курсов до прохождения адаптивного тестирования и анализа результатов. Для предметной области это означает сокращение ручных операций, повышение прозрачности контроля обучения и возможность дальнейшего развития системы под задачи конкретной организации.

Структура работы соответствует поставленным задачам и включает введение, три главы, заключение, список использованных источников из 51 наименования и пять приложений. Объём основного текста (без титульного листа, задания на ВКР, аннотации и приложений) составляет порядка 70–85 страниц формата А4; иллюстративный материал включает более 15 рисунков и более 10 таблиц — точные значения определяются после финальной верстки PDF.

Во введении обоснованы актуальность темы, определены цель, задачи, объект, предмет, методы и информационная база исследования, описан практический результат. В первой главе рассматриваются предметная область корпоративного обучения, постановка проблемы, требования к системе, сравнительный анализ аналогов с весами критериев и обоснование выбора технологий. Во второй главе описываются архитектура модульного монолита, пользовательские сценарии, инфологическая и даталогическая модели данных, проектирование интерфейсов, реализация клиентских приложений и серверной части, алгоритмы адаптивного тестирования и изоляции данных арендаторов. В третьей главе приводятся развёртывание программного комплекса, план и результаты опытной эксплуатации и испытаний, анализ информационной безопасности, экономическое обоснование и маркетинговая

стратегия развития продукта. Список использованных источников оформлен по ГОСТ с использованием стиля gost780u.[1, 13]

1 ПРЕДМЕТНАЯ ОБЛАСТЬ И ТЕХНОЛОГИИ

1.1 Предметная область корпоративного обучения и исходное состояние

Системы управления обучением применяются для централизованного хранения учебных материалов, назначения курсов, контроля прохождения и фиксации результатов. В корпоративной среде LMS используются для адаптации новых сотрудников (онбординга), обязательного обучения, внутренних регламентов, курсов по информационной безопасности и повышения квалификации сотрудников.[31, 32]

Для рассматриваемой предметной области характерно наличие нескольких устойчивых ролей. Системный администратор поддерживает общесистемные настройки и контролирует работу платформы. Администратор организации управляет пользователями, ролями, курсами и назначениями в пределах своей организации. Преподаватель создаёт учебный контент, тесты и анализирует результаты. Слушатель проходит назначенные курсы, изучает уроки, выполняет тесты и получает рекомендации по слабым темам.

В исходном состоянии процесс обучения часто строится с использованием нескольких несвязанных инструментов: документы и видео хранятся отдельно, списки обучающихся ведутся вручную, результаты тестирования не агрегируются, а повторение слабых тем организуется без формальной аналитики. Это приводит к ряду практических проблем:

- а) отсутствует единый источник данных о пользователях, курсах и результатах;
- б) повышаются трудозатраты на назначение курсов и контроль прохождения;
- в) преподаватель не получает целостной картины по ошибкам и слабым темам;

г) слушатель проходит одинаковые по сложности тесты вне зависимости от уровня подготовки;

д) использование мобильных устройств поддерживается ограниченно или неудобно.

На рисунке 1 представлена схема процесса корпоративного обучения в цифровой среде. Диаграмма показывает последовательность действий от подготовки контента и назначения курса до прохождения тестирования и анализа результатов.

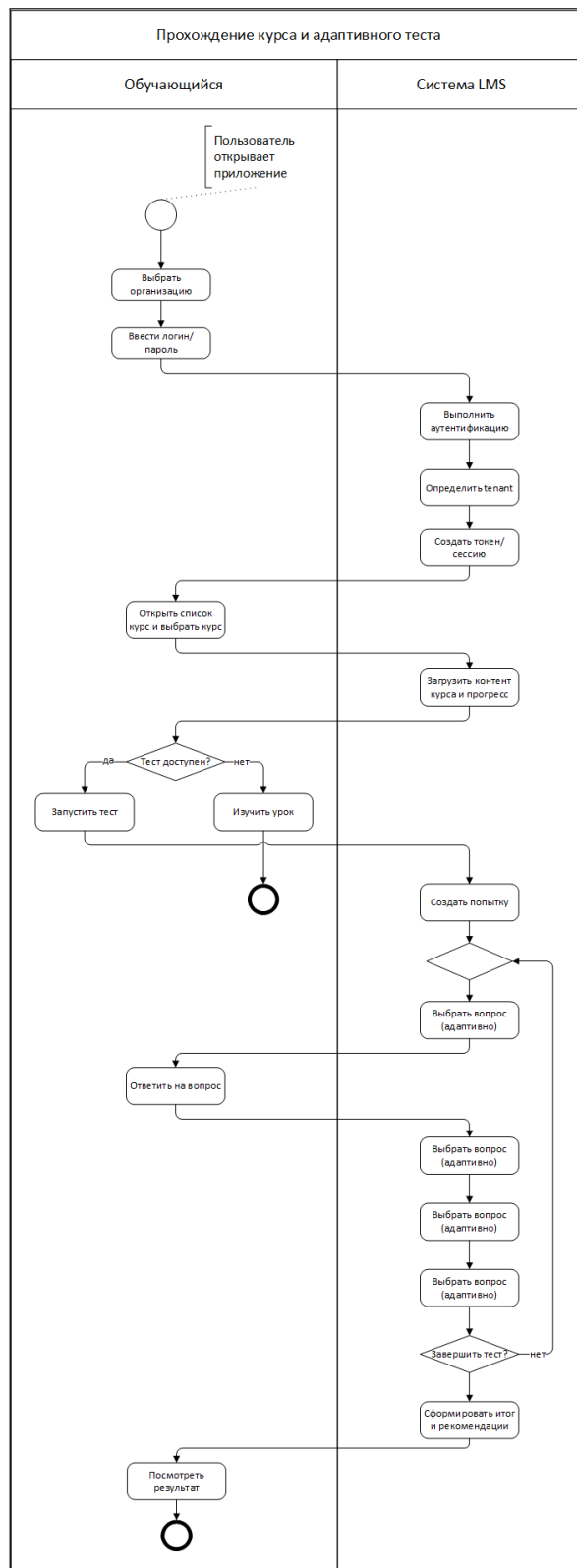


Рисунок 1 – Схема процесса корпоративного обучения в цифровой среде

1.2 Постановка проблемы и назначение системы

Проблема разработки состоит в необходимости объединить в одной системе функции администрирования, хранения контента, назначения учеб-

ных программ, мобильного доступа, тестирования и анализа результатов без усложнения пользовательских сценариев. Типовые LMS нередко обладают избыточным или трудно настраиваемым функционалом, а их мобильный пользовательский опыт не всегда соответствует сценарию коротких учебных сессий.[33, 8]

Назначение разрабатываемой системы заключается в поддержке полного цикла корпоративного обучения:

- а) создание и редактирование пользователей, ролей, курсов, уроков и тестов;
- б) назначение курсов слушателям;
- в) прохождение уроков и фиксирование прогресса;
- г) запуск адаптивного теста и подбор вопросов с учётом текущей сложности;
- д) формирование рекомендаций по слабым темам;
- е) просмотр аналитики по обучению в административном интерфейсе.

Система должна обеспечивать работу нескольких организаций в одном развёртывании. В проекте эта задача решается не путём создания отдельной базы данных на каждого арендатора, а через логическую мультиарендную модель (несколько организаций в одном экземпляре): идентификация организации в запросе, проверка участия пользователя в организации и фильтрация данных по полю `tenant_id`. Такой подход достаточен для учебного минимально жизнеспособного продукта (MVP) и соответствует текущей реализации проекта.

На рисунке 2 показана диаграмма вариантов использования, отражающая ключевые действия ролей в системе. Диаграмма фиксирует распределение функций между системным администратором, администратором организации, преподавателем и слушателем.

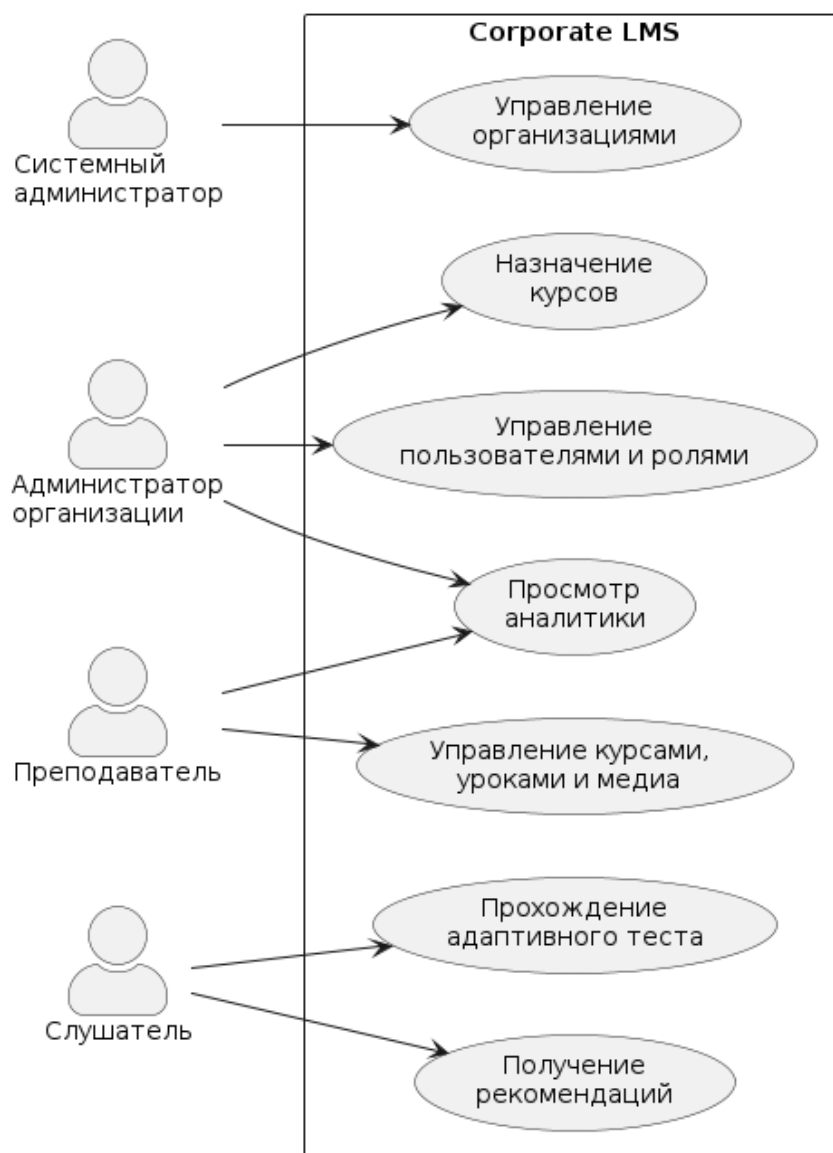


Рисунок 2 – Диаграмма вариантов использования системы

Для более формального представления сценариев в таблице 1 приведены основные прецеденты системы, их акторы и ожидаемые результаты выполнения.

Таблица 1 – Ключевые прецеденты системы

Прецедент	Основной актер	Результат выполнения
Управление организациями	Системный администратор	Созданы и настроены организации, доступные для дальнейшей работы пользователей.
Управление пользователями и ролями	Администратор организации	В организации созданы пользователи и назначены роли в соответствии с полномочиями.
Управление курсами, уроками и медиа	Преподаватель	Подготовлен учебный контент, доступный для назначения слушателям.
Назначение курса	Администратор организации	Выбранный курс назначен слушателю и появился в его списке обучения.
Прохождение адаптивного теста	Слушатель	Сформирована завершённая попытка с результатом и историей ответов.
Получение рекомендаций	Слушатель	Пользователь получает перечень тем и материалов для повторения.
Просмотр аналитики	Преподаватель, администратор организации	Получены агрегированные показатели по курсам, попыткам и успеваемости.

1.3 Требования к системе

Требования к программному продукту сформированы с учётом методических рекомендаций по ВКР и общепринятых принципов тестирования требований.[1, 12] В рамках анализа требований важно не только перечислить ожидаемую функциональность, но и проверить требования на завершённость, непротиворечивость, атомарность, недвусмысленность и проверяемость.[12]

1.3.1 Уровни и типы требований

Для разрабатываемой корпоративной LMS требования целесообразно рассматривать на нескольких уровнях: бизнес-требования, пользовательские требования и требования к продукту.[12] Такой подход позволяет связать цель проекта с ролями пользователей и с конкретной реализуемой функциональностью.

Бизнес-требование. Организация должна получить единый программный комплекс, обеспечивающий управление корпоративным обучением, на-

значение курсов, контроль прохождения, адаптивную проверку знаний и анализ результатов без использования набора разрозненных инструментов.

Пользовательские требования. На пользовательском уровне требования фиксируют задачи, которые выполняют роли системы, и поэтому в работе представлены в виде вариантов использования, табличных прецедентов и формализованных сценариев.

а) системный администратор должен иметь возможность управлять организациями и общесистемными настройками;

б) администратор организации должен иметь возможность управлять пользователями, ролями и назначениями курсов в пределах своей организации;

в) преподаватель должен иметь возможность создавать курсы, уроки, тесты и анализировать результаты обучения;

г) слушатель должен иметь возможность проходить назначенные курсы, выполнять тестирование и получать рекомендации по слабым темам.

Требования к продукту. На уровне продукта требования разделяются на функциональные и нефункциональные. Дополнительно в проекте выделяются требования к интерфейсам, требования к данным и ограничения, так как именно эти группы напрямую влияют на архитектуру системы, структуру API и модель хранения данных.

1.3.2 Функциональные требования

Функциональные требования сгруппированы по подсистемам.

Подсистема аутентификации и выбора организации

а) Система должна обеспечивать регистрацию и аутентификацию пользователей.

б) Система должна поддерживать выдачу токенов доступа и обновления.

в) Система должна определять текущую организацию по контексту запроса и проверять, что пользователь имеет активное участие в выбранной организации.

Подсистема управления пользователями и ролями

а) Администратор организации должен иметь возможность создавать пользователей.

б) Администратор организации должен иметь возможность изменять состояние доступа пользователя.

в) Система должна поддерживать роли слушателя, преподавателя, администратора организации и системного администратора.

Подсистема управления курсами и уроками

а) Преподаватель и администратор должны иметь возможность создавать и редактировать курсы.

б) Преподаватель и администратор должны иметь возможность создавать и редактировать уроки.

в) Система должна поддерживать хранение структурированного контента урока, изображений и видео.

Подсистема тестирования

а) Преподаватель и администратор должны иметь возможность создавать тесты и вопросы.

б) Система должна поддерживать указание базовой сложности теста и лимита вопросов.

в) Система должна обеспечивать запуск попытки, выдачу следующего вопроса, приём ответа и завершение попытки.

Подсистема адаптивного тестирования и рекомендаций

а) После каждого ответа система должна изменять текущую сложность попытки на один уровень вверх или вниз.

б) Система должна выбирать следующий вопрос среди ещё не заданных вопросов, наиболее близких к текущей сложности.

в) Система должна выявлять слабые темы по ошибочным и медленным ответам.

г) После завершения теста система должна формировать рекомендации по повторению материала.

Подсистема аналитики

а) Система должна предоставлять агрегированные показатели по пользователям, курсам, тестам и назначенным обучением.

б) Система должна отображать проблемные темы и индивидуальные результаты слушателя.

в) Система должна обеспечивать просмотр истории попыток и детального разбора попытки.

1.3.3 Нефункциональные требования

Безопасность

а) Пароли пользователей должны храниться в виде криптографических хэшей.

б) Доступ к защищённым операциям должен выполняться только после аутентификации и проверки роли.

в) Система должна вести аудит ключевых действий пользователей.

Изоляция данных

а) Данные разных организаций не должны пересекаться при выполнении запросов.

б) Любая tenant-зависимая сущность должна быть связана с идентификатором организации.

Производительность и сопровождаемость

а) Время отклика на типовые операции чтения должно быть приемлемым для интерактивной работы пользователя.

б) Серверная часть должна быть структурирована по подсистемам и слоям, упрощающим сопровождение и развитие.

в) Система должна поддерживать контейнеризированное локальное развёртывание.

Удобство использования и кроссплатформенность

а) Веб-интерфейс должен быть ориентирован на административные сценарии.

б) Мобильный интерфейс должен быть ориентирован на сценарии слушателя.

в) Платформа должна поддерживать как минимум две клиентские платформы: web и mobile.

1.3.4 Требования к интерфейсам, данным и ограничения

В дополнение к функциональным и нефункциональным требованиям целесообразно зафиксировать связанные группы требований, уточняющие границы решения и способ взаимодействия подсистем.

Требования к интерфейсам

а) Взаимодействие web- и mobile-клиентов с серверной частью должно выполняться через REST API с передачей данных в формате JSON.

б) Защищённые операции должны выполняться только после аутентификации пользователя по токenu доступа.

в) Для tenant-зависимых операций система должна определять текущую организацию по контексту запроса и не допускать обращения к данным другой организации.

Требования к данным

а) Сущности, зависящие от организации-арендатора, должны быть связаны с идентификатором организации `tenant_id`.

б) Система должна хранить структуру курсов и уроков, вопросы тестов, историю попыток, результаты проверки, слабые темы и сформированные рекомендации.

в) Ролевая модель должна храниться отдельно от учетной записи пользователя и учитывать участие пользователя в одной или нескольких организациях.

Ограничения

а) Система разрабатывается как клиент–серверный программный комплекс, включающий серверную часть, web-клиент и mobile-клиент.

б) В текущей реализации изоляция данных арендаторов должна обеспечиваться логически в рамках общего экземпляра базы данных, а не путём выделения отдельной БД на каждого арендатора.

в) Локальное развёртывание должно быть воспроизводимым и выполняться средствами контейнеризации.

1.3.5 Свойства качественных требований

С точки зрения методики анализа требований требования к системе должны удовлетворять базовым качественным свойствам.[12]

Таблица 2 – Проверка набора требований на свойства качества

Свойство	Применение к требованиям системы
Завершённость	Для каждой подсистемы указаны действия пользователя и ожидаемый результат системы.
Атомарность	Требования сформулированы как отдельные проверяемые утверждения без объединения разных сценариев в одном пункте.
Непротиворечивость	Роли и права доступа согласованы с фактической моделью RBAC, реализованной в проекте.
Недвусмысленность	Избегаются выражения вида “удобный”, “быстрый”, “интеллектуальный” без контекста проверки.
Прослеживаемость	Требования связаны с ролями, прецедентами, критическим путём пользователей и архитектурными решениями, что позволяет проследить путь от цели системы до проверки реализации.
Модифицируемость	Требования сгруппированы по подсистемам и типам, что упрощает их расширение, корректировку и локализацию изменений.
Выполнимость	Формулировки ограничены фактическими возможностями текущего проекта, используемым стеком и принятой логической мультиарендной моделью.
Обязательность и актуальность	В перечень включены только требования, непосредственно связанные с целью ВКР, ролями системы и критическим пользовательским сценарием.
Корректность	Требования описывают поведение системы и её свойства, а не действия пользователя, и не содержат заявлений о функциях, отсутствующих в реализации.
Проверяемость	Для критических функций возможно построить тестовые сценарии и получить однозначный результат проверки.

1.3.6 Сценарий тестирования требований по критическому пути пользователей

В качестве критического пути пользователей выбран сквозной сценарий, затрагивающий ключевые роли и основные бизнес-функции системы. Такой сценарий позволяет связать требования, архитектурные решения и проверку реализации в единую последовательность действий.

- а) преподаватель или администратор организации входит в систему;
- б) преподаватель создаёт курс, уроки и тест;
- в) администратор назначает курс слушателю;
- г) слушатель проходит урок и запускает адаптивный тест;
- д) система фиксирует попытку, изменяет сложность вопросов, завершает тест и формирует рекомендации;

е) администратор или преподаватель просматривает аналитику и результаты слушателя.

Данный сценарий позволяет проверить полноту и согласованность требований, поскольку в нём задействованы создание контента, назначение обучения, прохождение курса, серверная логика тестирования, рекомендации и аналитика. В таблице 3 приведена матрица трассировки критического пути, связывающая шаги сценария с группами требований и ожидаемыми результатами проверки.

Таблица 3 – Матрица трассировки критического пути пользователей

Шаг	Роль	Проверяемые требования	Ожидаемый результат
1	Преподаватель или администратор организации	Аутентификация, выбор организации, разграничение прав доступа	Пользователь получает доступ к функциям своей роли в пределах выбранной организации.
2	Преподаватель	Управление курсами, уроками, тестами и медиа	Учебный контент создан и подготовлен к назначению слушателю.
3	Администратор организации	Назначение курса, tenant-изоляция и контроль ролей	Выбранный курс становится доступным назначенному слушателю.
4	Слушатель	Просмотр уроков, запуск теста, выполнение попытки	Слушатель проходит урок и запускает адаптивное тестирование.
5	Система	Изменение сложности, фиксация попытки, выявление слабых тем, формирование рекомендаций	Сохраняются результат, история ответов, слабые темы и рекомендации по повторению материала.
6	Преподаватель или администратор организации	Аналитика, история попыток, детализированный просмотр результатов	Административная роль получает агрегированные и детализированные данные по обучению.

На рисунке 3 показана иерархия ключевых требований системы, используемая как схема вертикальной трассировки от цели проекта к основным группам функций.

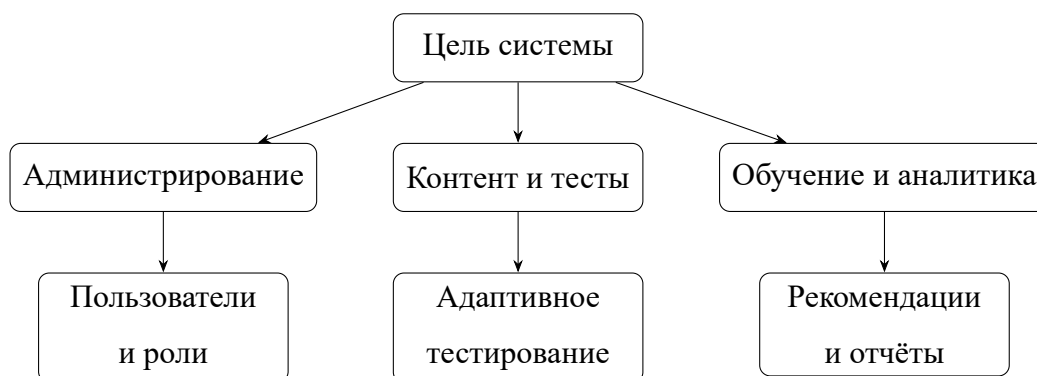


Рисунок 3 – Иерархия ключевых требований системы

1.4 Анализ аналогов

Для сравнения выбраны решения, представляющие разные классы LMS: открытые платформы (*Moodle*, *Open LMS*), облачный вариант на базе Moodle (*MoodleCloud*), платформы массовых онлайн-курсов и коммерческие корпоративные LMS.[31, 32, 34, 35]

Сравнение выполняется по критериям, значимым для данной ВКР:

- а) наличие административных ролей и разделения прав доступа;
- б) удобство мобильного доступа для слушателя;
- в) наличие или отсутствие адаптивного тестирования;
- г) возможности аналитики и работы со слабыми темами;
- д) поддержка мультиарендного сценария (несколько организаций в одном развёртывании);
- е) пригодность для учебной разработки и доработки архитектуры.

Для формализации сравнения в таблице 4 заданы критерии и удельные веса w_i такие, что $\sum_i w_i = 1$. Баллы x_{ij} по 10-балльной шкале (экспертные оценки по материалам документации и опросным листам приложения В) сведены в таблице 5. Итоговый показатель для варианта j вычисляется как взвешенная сумма[13]

$$S_j = \sum_i w_i x_{ij}. \quad (1)$$

Результаты свёртки приведены в таблице 6. Наивысший показатель у собственной разработки, что согласуется с целью ВКР — продемонстрировать архитектуру, модель данных и алгоритмы в открытом для анализа коде; коммерческие LMS (в таблице представлены iSpring Learn и GetCourse как ориентиры сегмента) получают высокие баллы по готовому функционалу, но низкие по критерию прозрачности архитектуры для учебного проекта. Численные оценки x_{ij} — ориентировочные и подлежат уточнению при заполнении приложения В.

Таблица 4 – Критерии сравнения LMS-аналогов и удельные веса

Критерий i	Содержание	Вес w_i
1	Административные роли и RBAC	0,15
2	Мобильный доступ для слушателя	0,20
3	Адаптивное / персонализированное тестирование	0,25
4	Аналитика и слабые темы	0,15
5	Мультиарендность	0,15
6	Пригодность для анализа архитектуры в ВКР	0,10
Сумма весов		1,00

Таблица 5 – Оценка аналогов по критериям (шкала 1–10, x_{ij})

Критерий	Moodle	Open LMS	MoodleCloud	iSpring	GetCourse	Собственная
1	8	8	7	8	8	8
2	6	6	6	8	8	8
3	5	5	5	7	7	9
4	6	6	6	8	8	8
5	5	5	5	7	7	8
6	4	4	4	4	4	9

Таблица 6 – Свёртка оценок по аналогам (итоговый балл S_j по формуле 1)

Решение j	S_j (условные единицы)
Moodle	5,70
Open LMS	5,70
MoodleCloud	5,55
iSpring Learn (ориентир)	7,20
GetCourse (ориентир)	7,20
Собственная разработка	8,35

Поскольку административные роли поддерживаются всеми рассматриваемыми решениями, в таблице 7 дополнительно приведено качественное сопоставление по ключевым признакам, по которым платформы различаются наиболее заметно для пользователя.

Таблица 7 – Сравнение аналогов по ключевым критериям

Решение	Мобильный доступ	Адаптивное тестирование	Аналитика	Мультиарендность
Moodle	средне	ограничено	да	ограничено
Open LMS	средне	ограничено	да	ограничено
MoodleCloud	средне	ограничено	да	ограничено
Коммерческие LMS	хорошо	частично или да	хорошо	да
Собственная система	хорошо	да	да	да

По результатам сравнения можно сделать следующие выводы. Moodle и производные от него платформы обеспечивают широкий базовый функционал, однако требуют более сложной настройки и не ориентированы на демонстрацию собственной архитектуры в рамках учебной разработки. Коммерческие LMS предлагают зрелые средства мобильного доступа и аналитики, но являются закрытыми и хуже подходят как основа выпускного проекта. Разрабатываемая система уступает крупным платформам по степени зрелости, но лучше соответствует целям ВКР, поскольку позволяет явно продемонстрировать архитектуру, модель данных, адаптивное тестирование и изоляцию данных организаций-арендаторов.

Результаты сравнения показывают, что существующие решения закрывают базовые задачи LMS, однако либо обладают избыточностью и сложностью настройки, либо не ориентированы на исследование и реализацию собственной архитектуры. Для целей ВКР целесообразно разрабатывать собственный программный комплекс, в котором можно явно продемонстрировать клиент–серверную архитектуру, модель данных, алгоритм адаптивного тестирования и изоляцию данных арендаторов.

1.5 Выбор технологий и платформ

Выбор технологий определялся не только их распространённостью, но и соответствием требованиям проекта. Разрабатываемая система должна поддерживать отдельные web- и mobile-клиенты, REST API, изоляцию дан-

ных организаций-арендаторов, развитие модели предметной области и воспроизводимое локальное развёртывание. Поэтому для каждой подсистемы рассматривались несколько вариантов, после чего выбиралось решение, лучше всего соответствующее целям ВКР и текущему масштабу проекта.

1.5.1 Критерии выбора технологий

При выборе технологического стека учитывались следующие критерии:

- а) соответствие клиент–серверной архитектуре с отдельными web- и mobile-клиентами;
- б) удобство реализации REST API, аутентификации, разграничения доступа и контекста организации-арендатора;
- в) возможность формального описания модели данных и сопровождения изменений схемы БД;
- г) пригодность для минимально жизнеспособного продукта (MVP), который необходимо не только разработать, но и продемонстрировать на защите;
- д) воспроизводимость локального развёртывания без сложной инфраструктуры.

1.5.2 Сравнение технологий и результат выбора

Сравнение основных вариантов приведено в таблице 8. В таблицу включены именно те подсистемы, которые определяют архитектурный облик проекта и влияют на удобство дальнейшей доработки.

Таблица 8 – Сравнение технологий и результаты выбора

Подсистема	Рассмотренные варианты	Выбранная технология и причина
Серверная часть	FastAPI, Django REST Framework, Flask	Выбран FastAPI, так как проект имеет формат API-first и не требует серверной генерации HTML. По сравнению с Django REST Framework решение получается легче и проще для MVP, а по сравнению с Flask требует меньше ручной сборки маршрутов, схем и документации.
Доступ к данным	SQLAlchemy + Alembic, Django ORM, прямой SQL	Выбраны SQLAlchemy и Alembic, поскольку они не привязаны к конкретному веб-фреймворку, позволяют явно описывать модели и контролировать миграции. Прямой SQL усложнил бы сопровождение, а Django ORM логичнее использовать только вместе с Django.
Web-клиент	React + TypeScript, Vue, Angular	Выбраны React и TypeScript. Для административной панели важны компонентный подход, маршрутизация, работа с формами и типизация данных. Angular для MVP избыточен, а Vue также подходит, но в текущем проекте более удобным оказался гибкий компонентный подход React.
Мобильный клиент	Flutter, React Native, native Android	Выбран Flutter, поскольку он позволяет поддерживать единый код мобильного приложения и быстро собрать интерфейс слушателя. Нативная разработка увеличила бы трудоёмкость, а React Native не давал принципиального преимущества при уже разделённых web- и mobile-сценариях.
Хранение данных	SQLite, PostgreSQL, MySQL	Используется комбинированный подход: SQLite удобен для локальной разработки и тестирования без отдельного сервера, а PostgreSQL лучше подходит для контейнеризованного запуска и демонстрации многопользовательской системы.
Развёртывание	ручной запуск сервисов, Docker Compose, Kubernetes	Выбран Docker Compose, так как он достаточен для MVP и позволяет поднимать серверную часть, web и базу данных одной конфигурацией. Ручной запуск хуже воспроизводим, а Kubernetes для учебного проекта избыточен.

1.5.3 Обоснование выбора серверной части

Для серверной части рассматривались три основных подхода: использование полноценного фреймворка Django с Django REST Framework, минималистичного Flask и API-ориентированного FastAPI. В рамках данной ВКР сервер выполняет роль централизованного REST API для двух клиентов и не требует шаблонной серверной генерации страниц. По этой причине

возможности Django, связанные с административной панелью, шаблонами и встроенной ORM, для проекта оказались избыточными. Flask, напротив, предоставляет слишком низкий уровень абстракции, из-за чего значительную часть инфраструктурного кода пришлось бы собирать вручную.[26, 36]

FastAPI оказался наиболее сбалансированным вариантом. Он хорошо подходит для явного описания маршрутов, схем запросов и ответов, зависимостей и проверки входных данных. Это соответствует фактической реализации серверной части, где сервер разделён на группы маршрутов `auth`, `tenants`, `users`, `courses`, `media`, `tests`, `recommendations` и `analytics`. Такая структура хорошо сочетается с модульным монолитом и позволяет расширять систему без переработки базовой архитектуры.[26]

Для доступа к данным выбраны SQLAlchemy и Alembic.[30] Такой набор позволяет описывать сущности предметной области на уровне моделей, а изменения схемы фиксировать через миграции. Для ВКР это важно, поскольку система включает достаточно большую группу взаимосвязанных сущностей: организации, пользователей, роли, курсы, уроки, тесты, попытки, рекомендации и аналитические данные. Использование прямого SQL на всём протяжении проекта усложнило бы сопровождение и повторяемость изменений, тогда как связка SQLAlchemy + Alembic обеспечивает более управляемое развитие схемы.

1.5.4 Обоснование выбора технологий клиентских приложений

В административной части проекта рассматривались React + TypeScript, Vue и Angular. Для web-панели требуются страницы авторизации, управления пользователями, курсами, уроками, тестами, назначениями и аналитикой. Такой интерфейс состоит из большого числа форм, таблиц и маршрутов, которые должны переиспользовать общие компоненты и работать с типизированными данными. По этой причине выбор был сделан

в пользу React как гибкой компонентной библиотеки, а TypeScript был добавлен для уменьшения количества ошибок при работе с API-моделями, ролями, курсами и результатами тестирования.[37, 38]

Angular предоставляет более жёсткую архитектурную модель, однако для MVP это увеличивает объём шаблонного кода и порог входа. Vue также является сильным кандидатом, но в рассматриваемом проекте приоритет был отдан экосистеме React, которая хорошо сочетается с компонентной структурой административной панели, маршрутизацией и быстрым циклом сборки через Vite. Таким образом, выбор React + TypeScript оказался наиболее удобным именно для текущего состава интерфейсных сценариев.

Для мобильного клиента рассматривались Flutter, React Native и нативная Android-разработка. Так как в рамках ВКР необходимо было показать самостоятельный mobile-клиент слушателя, поддерживающий авторизацию, просмотр курсов, воспроизведение медиа, прохождение тестов и получение рекомендаций, требовалось решение с единым кодом приложения и предсказуемой сборкой интерфейса. Flutter отвечает этим требованиям и позволяет реализовать мобильный интерфейс как самостоятельную часть комплекса, не смешивая его с web-клиентом.[28]

Нативная Android-разработка на Kotlin обеспечила бы высокий уровень платформенной интеграции, но заметно увеличила бы трудоёмкость проекта. React Native позволил бы использовать близкий по идеологии стек, однако не снимал бы задачи отдельной мобильной адаптации и не давал бы существенного выигрыша по сравнению с уже выбранным web-стеком. Поэтому для данной ВКР Flutter оказался более рациональным выбором.

1.5.5 Выбор СУБД и средств развёртывания

На уровне хранения данных в проекте используется двухконтурный подход. Для локальной разработки и части тестовых сценариев применяется

SQLite, так как она не требует запуска отдельного сервера, упрощает старт проекта и ускоряет проверку изменений. Для контейнеризированного развёртывания используется PostgreSQL, так как серверная СУБД лучше отражает рабочий режим многопользовательской системы, где одновременно функционируют серверная часть, web и база данных.[39, 27]

Такое сочетание не является противоречием. Напротив, оно позволяет использовать простой режим разработки на ранних этапах и более реалистичный режим демонстрации на этапе развёртывания. Наличие в проекте конфигураций и для SQLite, и для PostgreSQL подтверждает, что выбранный стек ориентирован как на удобство разработки, так и на последующую демонстрацию результата.

Для локального развёртывания рассматривались ручной запуск сервисов, Docker Compose и более тяжёлые оркестрационные решения. Ручной запуск серверной части, web и базы данных неудобен для повторяемой демонстрации, так как требует последовательной настройки каждого компонента. Использование Kubernetes для учебного минимально жизнеспособного продукта (MVP) не даёт соразмерной выгоды и существенно усложняет инфраструктурную часть работы. По этой причине в проекте выбран Docker Compose, который позволяет в одном файле конфигурации описать сервисы базы данных, серверной части, web и при необходимости сервис начального заполнения данными.[40]

Таким образом, текущий стек выбран не случайно, а как результат сопоставления требований проекта и трудоёмкости реализации. Он соответствует фактической структуре репозитория, обеспечивает поддержку web- и mobile-платформ, удобен для развития предметной модели и позволяет воспроизводить демонстрировать программный комплекс в рамках ВКР.

1.6 Выводы по главе

В первой главе рассмотрена предметная область корпоративного обучения, выявлены проблемы исходного состояния процессов и сформулировано назначение разрабатываемой системы. Определены функциональные и нефункциональные требования, приведена их проверка на основные свойства качества, а также сформирован сценарий тестирования требований по критическому пути пользователей. Выполнено сравнение аналогов и обоснован выбор собственного программного решения. На основании требований и анализа существующих платформ определён технологический стек проекта, соответствующий фактической реализации и задачам выпускной квалификационной работы.

2 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ

2.1 Архитектура системы

Разрабатываемая система реализована как модульный монолит с REST API, объединённый в одном репозитории, но логически разделённый на серверную часть, веб-клиент и мобильный клиент. Такой подход упрощает сопровождение учебного проекта, позволяет явно показать взаимодействие подсистем и при этом сохраняет понятную декомпозицию по функциональным модулям.[41, 20, 22]

Серверная часть выступает единственным источником бизнес-логики и данных. Она отвечает за аутентификацию, выбор текущей организации, проверку ролей, работу с курсами и уроками, управление тестами, выполнение адаптивного тестирования, формирование рекомендаций и подготовку аналитики. Веб-клиент ориентирован на роли администратора и преподавателя. Мобильный клиент ориентирован на роль слушателя и закрывает сценарии прохождения обучения.

На рисунке 4 представлена контейнерная схема системы. Диаграмма показывает состав программного комплекса на уровне контейнеров и основные каналы взаимодействия между web-клиентом, mobile-клиентом, REST API серверной части, базой данных и хранилищем медиафайлов.[42]

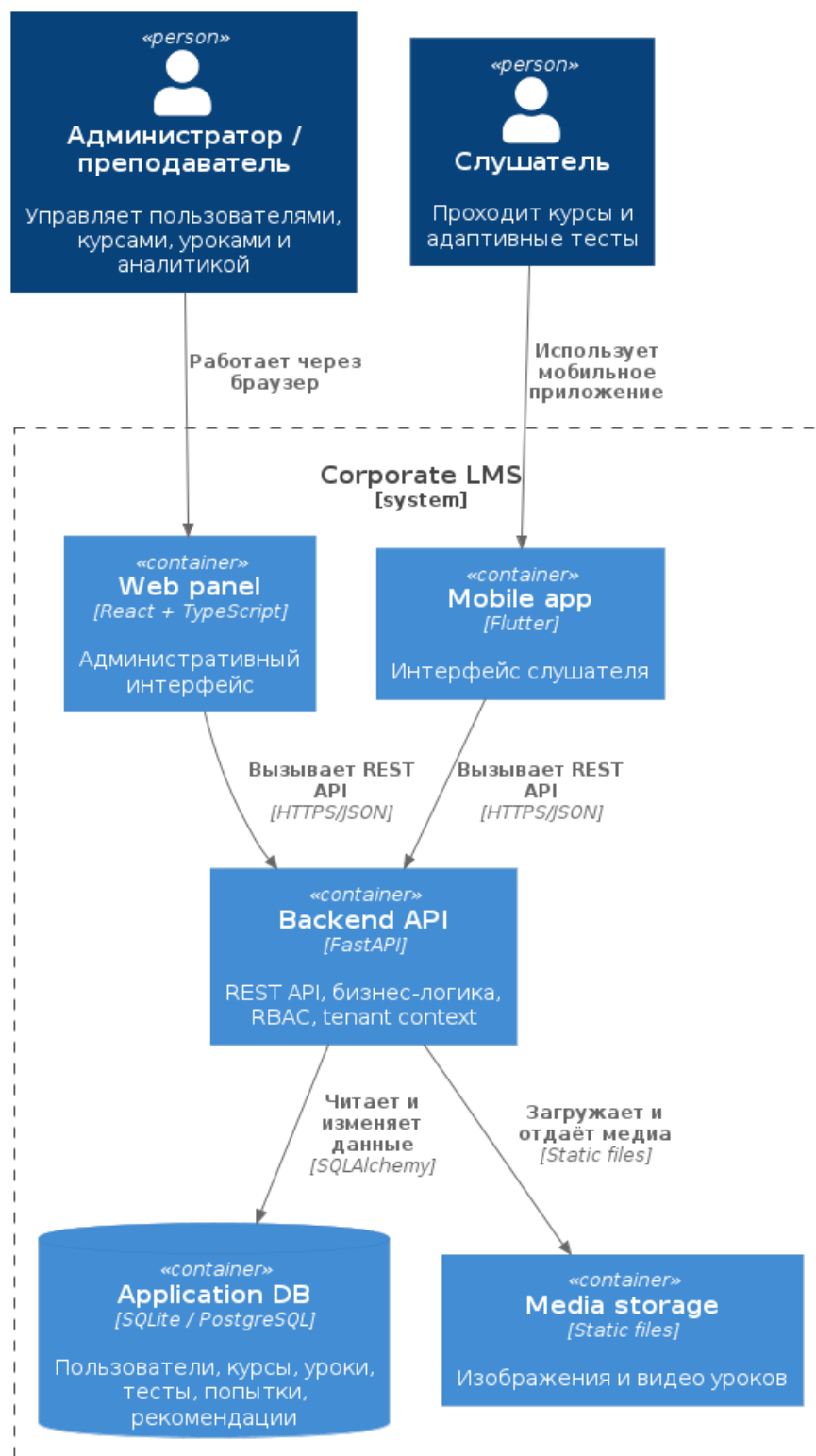


Рисунок 4 – Контейнерная схема программного комплекса

С точки зрения программной структуры серверная часть включает следующие подсистемы:[26, 30, 29]

- а) auth — аутентификация, токены доступа и профиль пользователя;
- б) tenants — работа с организациями и текущим контекстом организации-арендатора;

- в) `users` — управление пользователями и их состоянием;
- г) `courses` и `lessons` — управление курсами, уроками и прогрессом;
- д) `media` — загрузка и выдача медиафайлов;
- е) `tests` — жизненный цикл тестов и попыток;
- ж) `adaptive` — выбор следующего вопроса и расчёт рекомендаций;
- з) `recommendations` — выдача списка рекомендаций слушателю;
- и) `analytics` — агрегированная аналитика по системе.

На рисунке 5 приведена диаграмма компонентов серверной части. Она отражает декомпозицию серверного приложения на функциональные подсистемы и их связи с общими инфраструктурными механизмами.

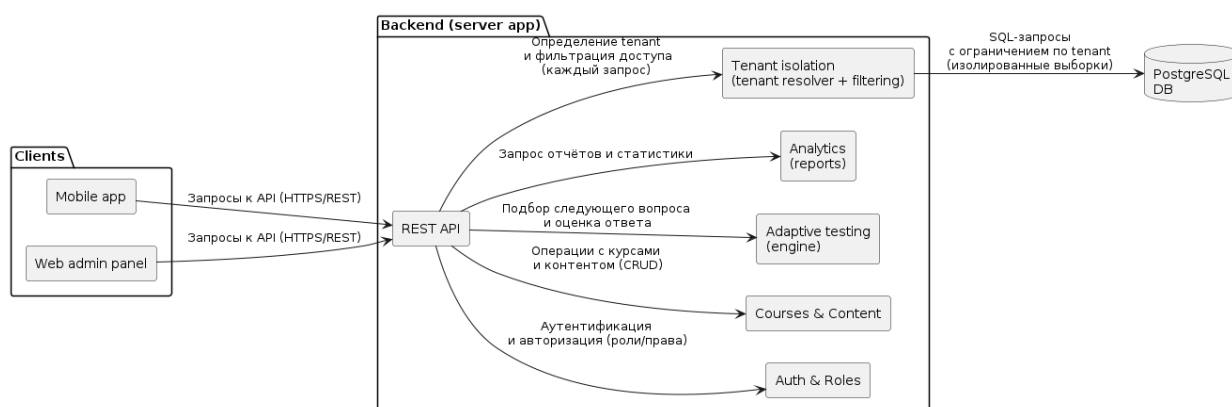


Рисунок 5 – Диаграмма компонентов системы

2.2 Сценарии использования

Во второй главе сценарии использования описываются как формальные последовательности действий, служащие основой для архитектурных и интерфейсных решений. Такое представление позволяет зафиксировать роли, предусловия, основной поток и результат выполнения для каждого ключевого сценария.[11, 43]

На основании диаграммы прецедентов и критического пути пользователей в таблице 9 приведено табличное представление ключевых сценариев, используемое для согласования ролей, предусловий и результатов выполнения.

Таблица 9 – Табличное представление основных сценариев использования

Сценарий	Актор	Предусловие	Результат
Подготовка обучения и назначение курса	Преподаватель, администратор организации	Пользователь аутентифицирован и работает в контексте своей организации	Курс создан, наполнен контентом и назначен слушателю.
Прохождение курса и адаптивного теста	Слушатель	Слушатель включён в организацию и имеет назначенный курс	Сформирован результат попытки и набор рекомендаций по слабым темам.
Анализ результатов обучения	Преподаватель, администратор организации	В системе накоплены данные по назначениям и попыткам	Получены агрегированные показатели и детализированные результаты по слушателям.

2.2.1 Сценарий 1: подготовка обучения и назначение курса

Акторы: администратор организации, преподаватель.

Предусловия: пользователь аутентифицирован, организация выбрана, роль пользователя позволяет управлять контентом и назначениями.

Основной поток:

- а) преподаватель создаёт курс;
- б) преподаватель добавляет уроки и при необходимости медиа-контент;
- в) преподаватель создаёт тест и вопросы;
- г) администратор организации выбирает слушателя;
- д) администратор назначает курс слушателю;
- е) система фиксирует назначение и делает курс доступным в мобильном клиенте слушателя.

Постусловия: слушатель получает назначенный курс и может приступить к изучению материала.

2.2.2 Сценарий 2: прохождение курса и адаптивного теста

Актор: слушатель.

Предусловия: слушатель аутентифицирован, имеет активное участие в организации, курс назначен, тест связан с курсом.

Основной поток:

- а) слушатель открывает список своих курсов;
- б) слушатель переходит в курс и изучает урок;
- в) слушатель запускает тест;
- г) система создаёт попытку с базовой сложностью;
- д) система выдаёт первый вопрос;
- е) слушатель отправляет ответ;
- ж) система оценивает ответ и изменяет текущую сложность;
- з) система выдаёт следующий вопрос с учётом новой сложности;
- и) после достижения лимита вопросов система завершает попытку;
- к) система сохраняет результат, слабые темы и формирует рекомендации.

Постусловия: результат попытки, история сложности и рекомендации доступны слушателю и административным ролям.

2.2.3 Сценарий 3: анализ результатов обучения

Акторы: преподаватель, администратор организации.

Основной поток:

- а) пользователь открывает раздел аналитики;
- б) система отображает агрегированные показатели по курсам, тестам и назначенным обучением;
- в) пользователь открывает проблемные темы или индивидуальный отчёт по слушателю;

г) система отображает результаты попыток, количество правильных ответов и рекомендации.

2.3 Модель данных

Модель данных построена вокруг сущностей, необходимых для поддержки мультиарендного обучения (несколько организаций в одном экземпляре).[21, 44, 45, 46] На концептуальном уровне она отражает объекты предметной области и их семантические связи, а на логическом уровне фиксирует состав реляционных таблиц, ключей и ограничений целостности. Центральной сущностью является организация (*tenant*), относительно которой изолируются пользователи, курсы, уроки, тесты, попытки и рекомендации.

2.3.1 Инфологическая модель базы данных

Инфологическая модель описывает систему без привязки к конкретной СУБД и показывает укрупнённые сущности предметной области: организацию, пользователя, курс, урок, тему, тест, вопрос, попытку и результат. Такая степень абстракции позволяет наглядно показать логику движения данных от подготовки учебного контента до прохождения адаптивного теста и получения результата.

На рисунке 6 приведена инфологическая модель базы данных. На этом уровне намеренно опущены служебные и ассоциативные сущности, связанные с ролями, назначениями, рекомендациями и аудитом, так как они раскрываются в даталогической модели.

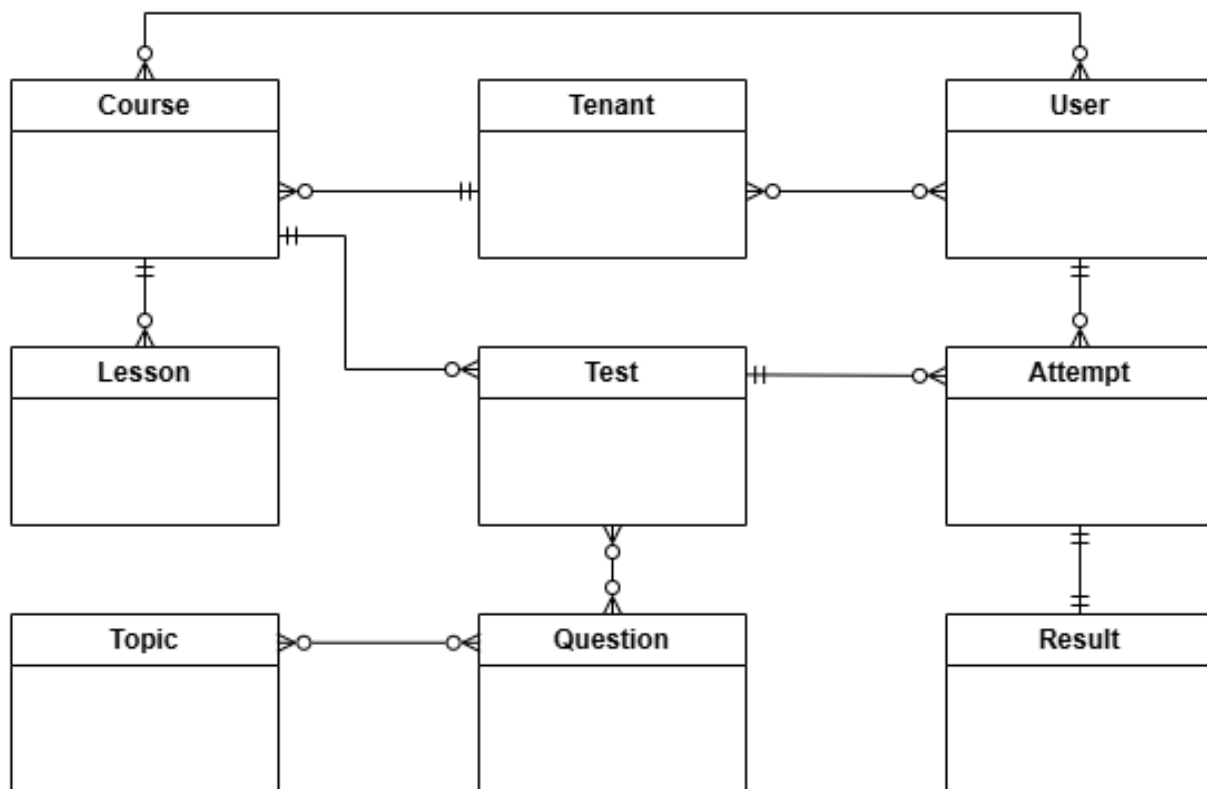


Рисунок 6 – Инфологическая модель базы данных

2.3.2 Даталогическая модель базы данных

На даталогическом уровне инфологическая модель преобразуется в реляционную схему, реализованную средствами SQLAlchemy. При локальном запуске приложение может использовать SQLite, а при контейнеризированном развёртывании — PostgreSQL; при этом набор таблиц, первичных и внешних ключей остаётся одинаковым. Для tenant-зависимых сущностей в схему включено поле `tenant_id`, а для критичных связей заданы ограничения уникальности, исключающие дублирование членств, назначений и записей о прогрессе.

Для пояснительной записки даталогическая модель представлена двумя основными фрагментами, отражающими базовые контуры проектируемой системы: организационный контур и контур управления учебным контентом. Такой способ позволяет показать структуру таблиц и связей без перегрузки рисунков второстепенными служебными сущностями.

На рисунке 7 показана даталогическая модель организационного контура. Она описывает, каким образом в системе представлены организации, пользователи, роли, членства и учебные группы. Данный фрагмент показывает реализацию мультиарендного подхода: один пользователь может состоять в нескольких организациях, а его права определяются через таблицу членства и связанную с ней роль. Дополнительно здесь отражено объединение пользователей в группы, используемые при массовом назначении обучения.

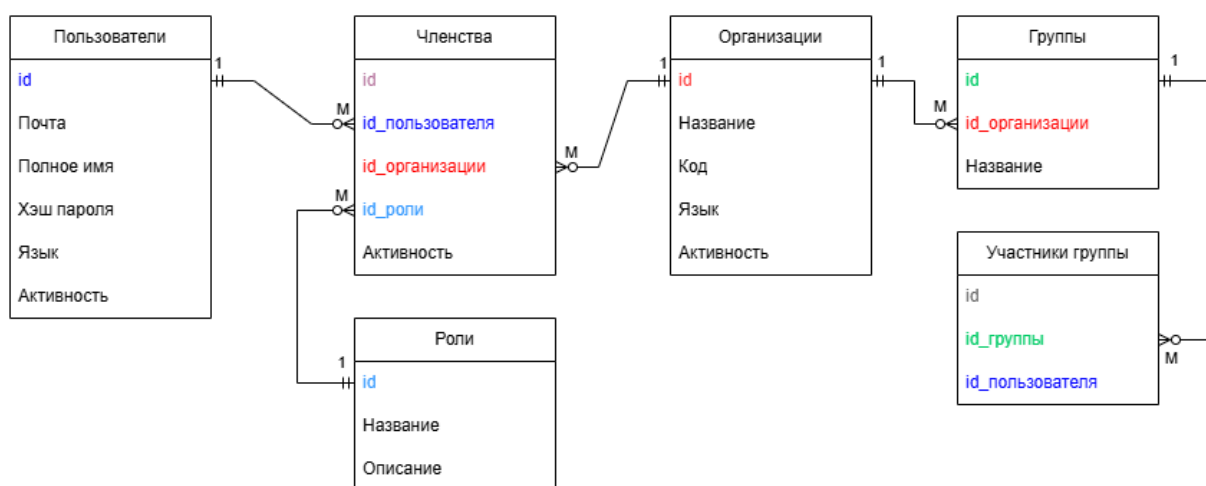


Рисунок 7 – Даталогическая модель организационного контура

На рисунке 8 представлена даталогическая модель контура курсов. В неё включены таблицы, описывающие учебный контент, структуру курса, назначения, зачисления и фиксацию прогресса. Схема показывает, что курс принадлежит организации, состоит из уроков и тем, может назначаться как отдельному пользователю, так и группе, а факт прохождения отражается в таблицах зачислений и прогресса по урокам. Такой контур обеспечивает хранение как структуры учебных материалов, так и состояния их освоения слушателями.

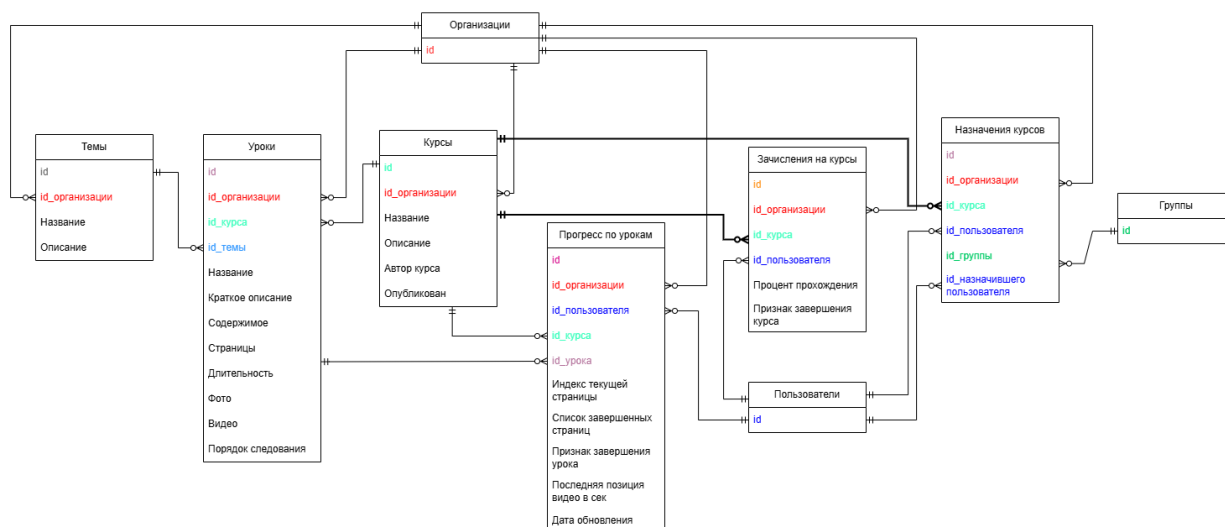


Рисунок 8 – Даталогическая модель контура курсов

Таким образом, в текущей версии пояснительной записки даталогическая модель раскрыта через два ключевых фрагмента: организационный и учебный. Первый описывает разграничение доступа и принадлежность пользователей к организациям и группам, а второй — структуру курсов, механизм их назначения и фиксацию прогресса обучения. Контуры тестирования, рекомендаций и служебных таблиц при необходимости могут быть представлены отдельно без перегрузки основной части работы.

2.4 Проектирование интерфейсов

Проектирование интерфейсов выполнялось отдельно для административного и пользовательского контуров с учётом принципов проектирования взаимодействия.[47, 48, 49]

2.4.1 Веб-интерфейс администратора и преподавателя

Веб-клиент предназначен для работы с учебным контентом и аналитикой. Его основными разделами являются:

- а) вход в систему и выбор организации;
- б) управление пользователями;

- в) управление курсами;
- г) управление уроками и медиатекой;
- д) управление тестами;
- е) назначение курсов;
- ж) просмотр аналитики;
- з) настройки профиля и языка интерфейса.

Структура веб-интерфейса ориентирована на последовательное выполнение административных операций: сначала создаётся контент, затем выполняются назначения, после чего доступны данные аналитики.

2.4.2 Мобильный интерфейс слушателя

Мобильный клиент ориентирован на компактный пользовательский сценарий:

- а) вход пользователя;
- б) просмотр списка назначенных курсов;
- в) открытие курса и урока;
- г) запуск теста;
- д) просмотр результата, рекомендаций и истории попыток.

На рисунках 9–10 показаны примеры экранов мобильного клиента.

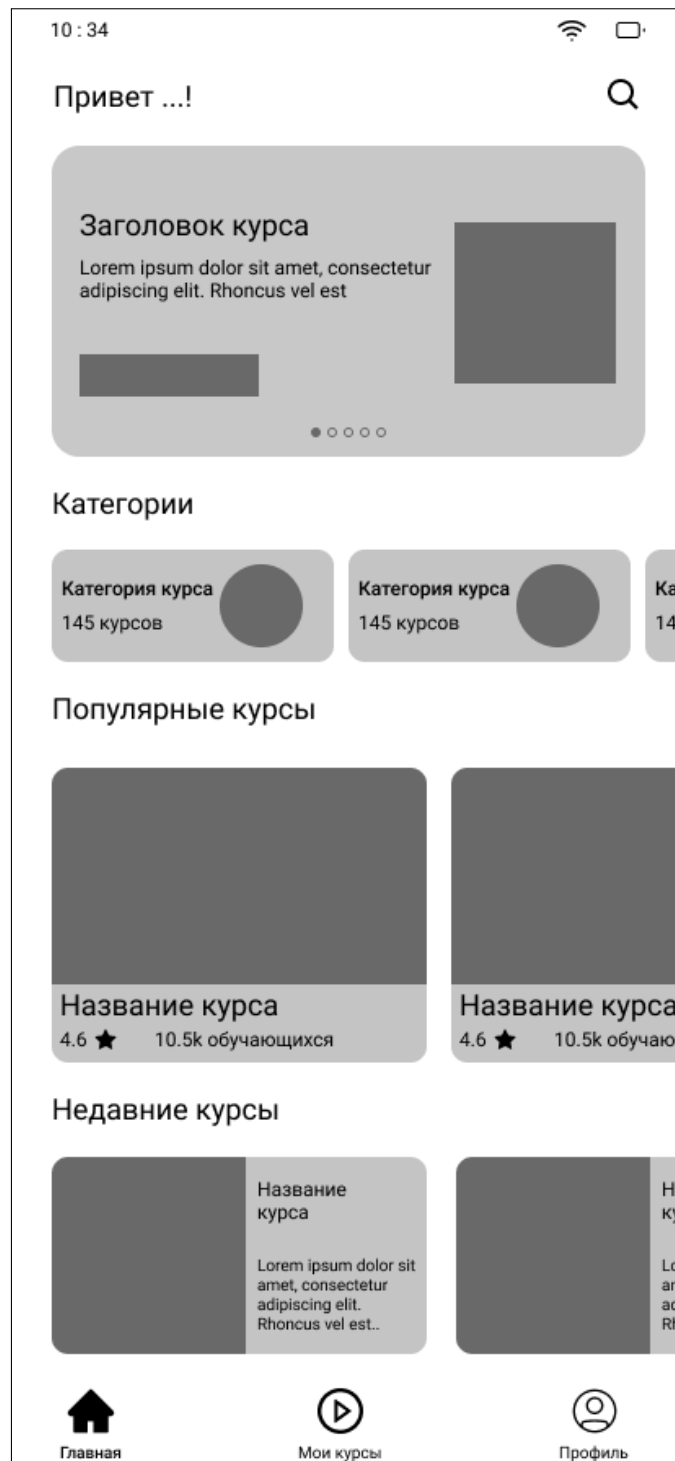


Рисунок 9 – Главный экран мобильного клиента

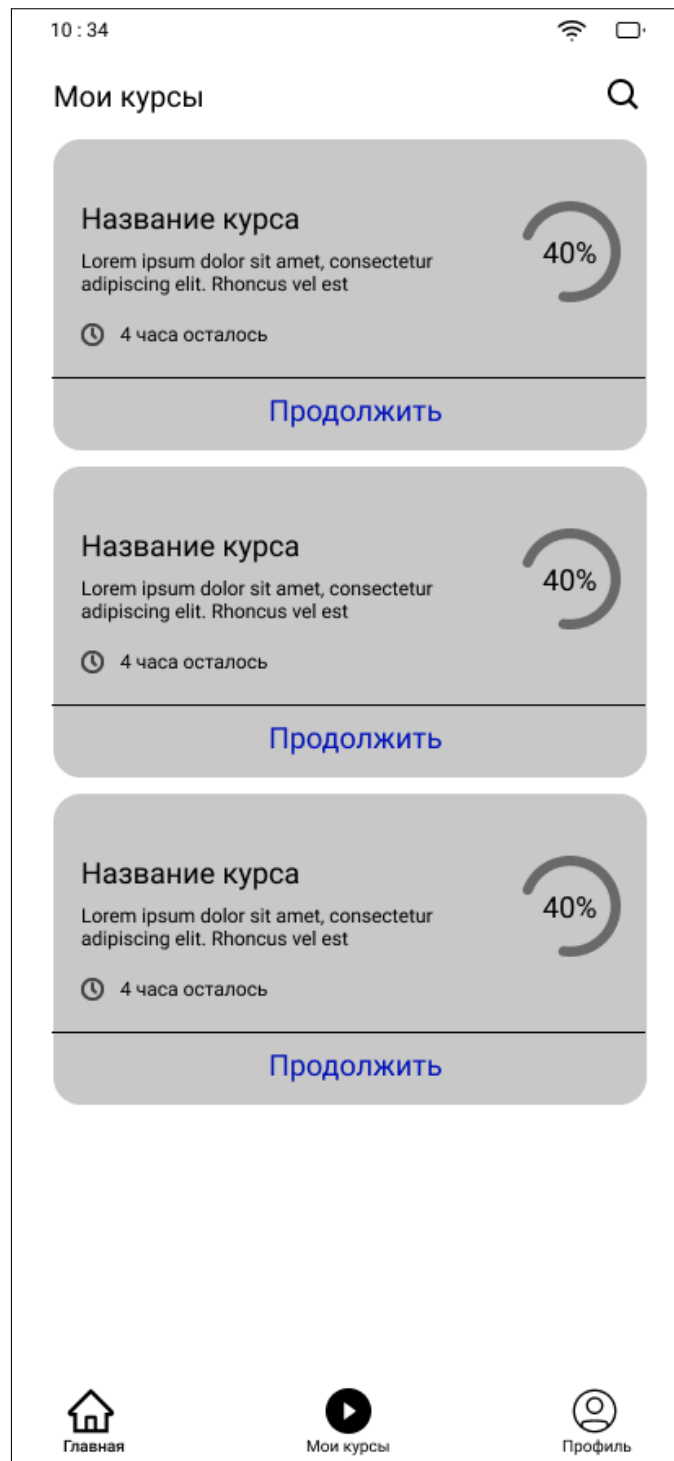


Рисунок 10 – Экран списка назначенных курсов

2.5 Реализация web- и mobile-клиентов

Клиентская часть системы реализована как два самостоятельных приложения, использующих единый REST API серверной части, но ориентированных на разные роли и сценарии работы. Веб-клиент предназначен для администратора и преподавателя, а мобильный клиент — для слушателя.

Такое разделение позволяет не перегружать интерфейсы лишними функциями и адаптировать взаимодействие с системой под реальные задачи пользователей.[37, 28]

На рисунке 11 представлена структурная схема клиентской части системы. Она показывает внутреннее устройство web- и mobile-приложений: пользовательский интерфейс, прикладные экраны, слой взаимодействия с API серверной части и локальное хранение пользовательского состояния. Общим для обоих клиентов является единый контракт обмена данными, а различие определяется набором экранов и способом управления состоянием.

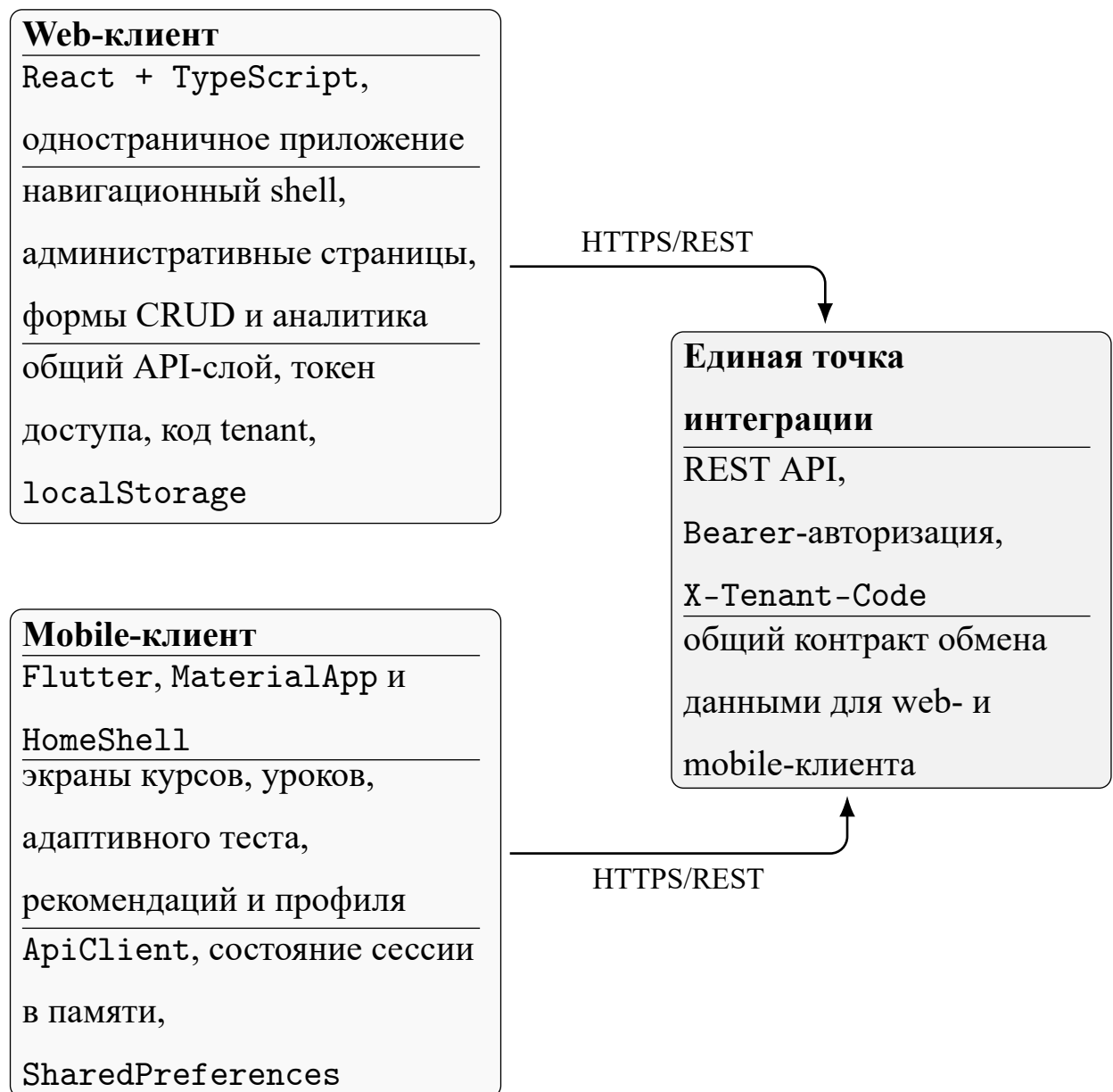


Рисунок 11 – Структурная схема клиентской части системы

2.5.1 Реализация web-клиента

Веб-клиент реализован как одностраничное приложение на React и TypeScript. После аутентификации пользователь попадает в общий каркас приложения с боковой навигацией, из которого доступны основные разделы: панель показателей, переключение организации, пользователи, курсы, уроки, тесты, назначения, аналитика и настройки. Маршрутизация организована на стороне клиента, что позволяет быстро переходить между разделами без повторной загрузки страницы.

Работа с серверной частью вынесена в отдельный API-слой. Для всех запросов централизованно задаются базовый адрес сервера, заголовок авторизации Bearer и заголовок X-Tenant-Code, определяющий текущую организацию. Такой подход уменьшает дублирование кода и обеспечивает единое поведение для операций чтения, создания, редактирования и загрузки медиафайлов. Для типовых запросов используются отдельные функции отправки GET, POST, PATCH и multipart/form-data.

Сессионное состояние web-клиента включает токен доступа и код текущей организации. Эти данные передаются между страницами через состояние верхнего уровня приложения. Дополнительно в интерфейсе реализовано переключение языка, причём выбранный язык сохраняется в localStorage браузера и применяется при следующем открытии приложения.

Функциональные страницы web-клиента построены по единому принципу: загрузка данных через общий механизм запросов, отображение списка сущностей, форма создания или редактирования и визуальное подтверждение результата операции. Так реализованы управление пользователями, курсами, уроками, тестами и назначениями. В разделе уроков предусмотрено редактирование структуры страниц урока и загрузка медиафайлов, а в разделе аналитики — загрузка агрегированных показателей, проблемных тем и данных по конкретному слушателю. В результате веб-клиент закрывает весь ад-

министративный контур системы и напрямую поддерживает сценарии, описанные в первой главе.

2.5.2 Реализация мобильного клиента

Мобильный клиент реализован на Flutter как отдельное приложение слушателя. Корневой экран приложения построен на базе `MaterialApp`, а после входа пользователь переходит в основной `HomeShell`, содержащий нижнюю навигацию между тремя разделами: курсы, рекомендации и профиль. Для переключения между разделами используется `IndexedStack`, что позволяет сохранять состояние экранов при переходах.

Взаимодействие с сервером инкапсулировано в классе `ApiClient`. Он отвечает за формирование HTTP-запросов, добавление заголовков авторизации и заголовка организации (`X-Tenant-Code`), обработку ошибок и преобразование JSON-ответов в модели приложения. Через этот клиент выполняются аутентификация, загрузка курсов, получение структуры курса, получение рекомендаций, работа с попытками тестирования и загрузка данных для разбора завершённых попыток.

Состояние мобильного приложения разделено на два уровня. Языковые настройки сохраняются в `SharedPreferences`, благодаря чему выбор русского или английского языка восстанавливается после перезапуска приложения. Сессионные данные текущего пользователя передаются между экранами в памяти приложения, что достаточно для учебного минимально жизнеспособного продукта (MVP) и упрощает жизненный цикл клиента.

Экран списка курсов загружает назначенные пользователю курсы и открывает детальный экран курса. На экране курса выполняется параллельная загрузка структуры курса и связанных с ним тестов, после чего слушателю становятся доступны три действия: продолжение обучения, запуск адаптивного теста и просмотр истории попыток. Для прохождения урока использует-

ся отдельный экран-плеер, который получает данные урока с сервера и сохраняет прогресс пользователя, включая текущую страницу, набор завершённых страниц и позицию воспроизведения видео.

Отдельный экран `TestFlowScreen` реализует клиентскую часть адаптивного тестирования. Он инициирует попытку, получает очередной вопрос, отправляет выбранный ответ с учётом времени реакции и завершает попытку после исчерпания набора вопросов. После завершения теста слушатель видит результат, слабые темы и рекомендации. Дополнительно в мобильном клиенте реализованы экран рекомендаций и экран истории попыток с возможностью открыть детальный разбор завершённого теста. Таким образом, мобильный клиент не ограничивается просмотром контента, а поддерживает полный пользовательский цикл слушателя: от изучения курса до анализа собственных результатов.

2.6 Реализация серверной логики и алгоритмов

2.6.1 Реализация API-подсистем

Серверная часть реализует REST API, сгруппированное по назначению маршрутов. Такое разбиение определяет формат взаимодействия клиентов с серверной частью и набор доступных операций.

- а) `/auth` — регистрация, вход, обновление токенов, получение профиля;
- б) `/tenants` — список доступных организаций, текущая организация, выбор организации;
- в) `/users` — просмотр, создание и изменение пользователей;
- г) `/courses` и `/lessons` — работа с курсами, уроками, прогрессом и назначениями;
- д) `/media` — библиотека и загрузка медиа;

- е) `/tests` и `/attempts` — тесты, вопросы, попытки, история и разбор;
- ж) `/recommendations` — рекомендации слушателя;
- з) `/analytics` — сводная и детальная аналитика.

Такое разбиение соответствует структуре кода серверной части и обеспечивает независимое развитие функциональных подсистем при сохранении единой точки входа.

На рисунке 12 приведена структурная схема серверной части системы. В отличие от общей диаграммы компонентов, эта схема акцентирует именно уровни обработки запроса: маршрутный слой, контур tenant-контекста и разграничения доступа, сервисный слой бизнес-логики и слой хранения данных.

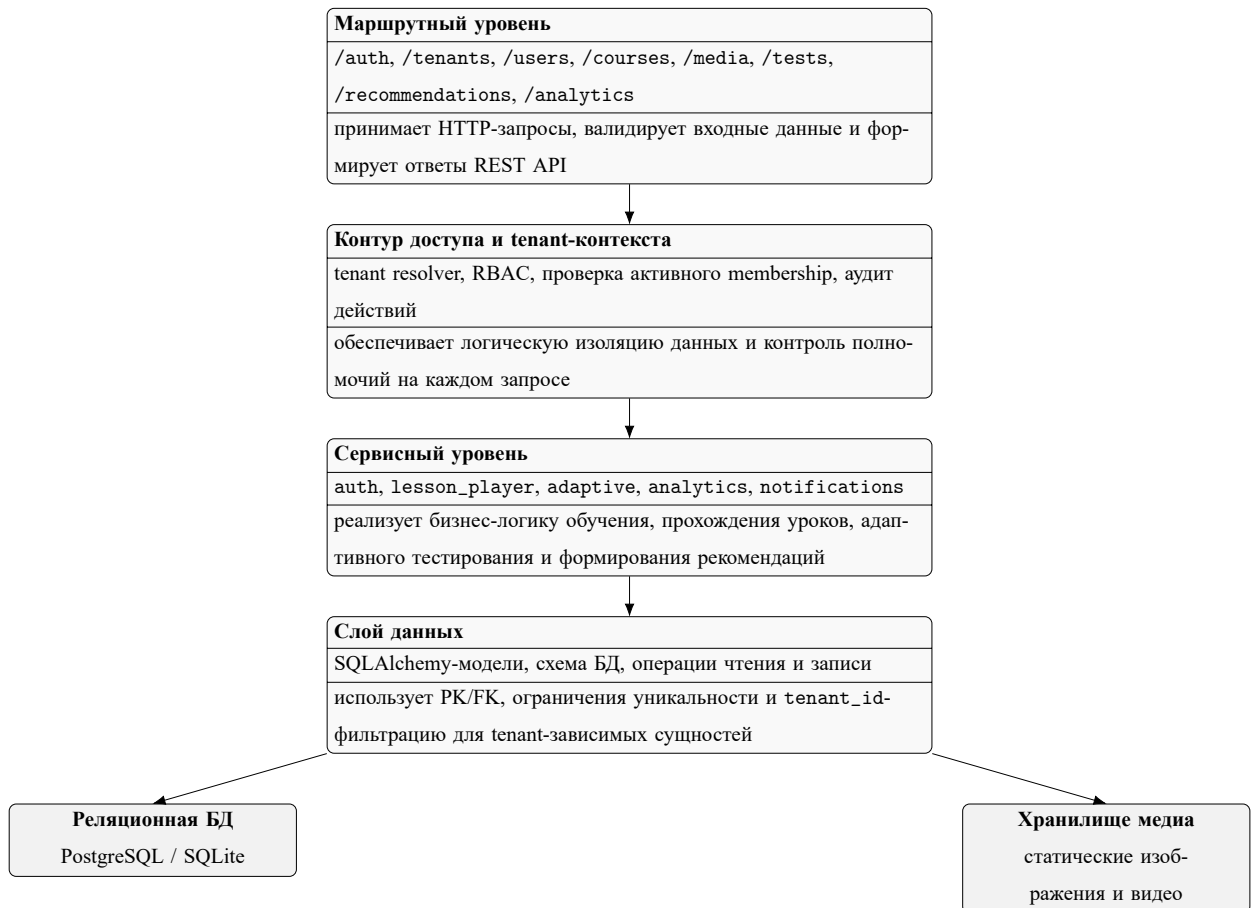


Рисунок 12 – Структурная схема серверной части системы

2.6.2 Алгоритм адаптивного тестирования

В проекте реализован дискретный алгоритм адаптивного тестирования, работающий в диапазоне сложностей от 1 до 5.[25, 5, 6] Алгоритм основан на следующих правилах:

- а) при старте попытки текущая сложность равна базовой сложности теста;
- б) после правильного ответа сложность повышается на один уровень;
- в) после неправильного ответа сложность понижается на один уровень;
- г) следующее задание выбирается среди ещё не заданных вопросов, наиболее близких к текущей сложности;
- д) по ошибочным и медленным ответам накапливаются слабые темы;
- е) после завершения попытки формируются рекомендации по повторению материала.

На рисунке 13 показана последовательность взаимодействия клиента и сервера при прохождении адаптивного теста. Диаграмма фиксирует обмен запросами между мобильным приложением, REST API серверной части и модулем адаптивной логики.

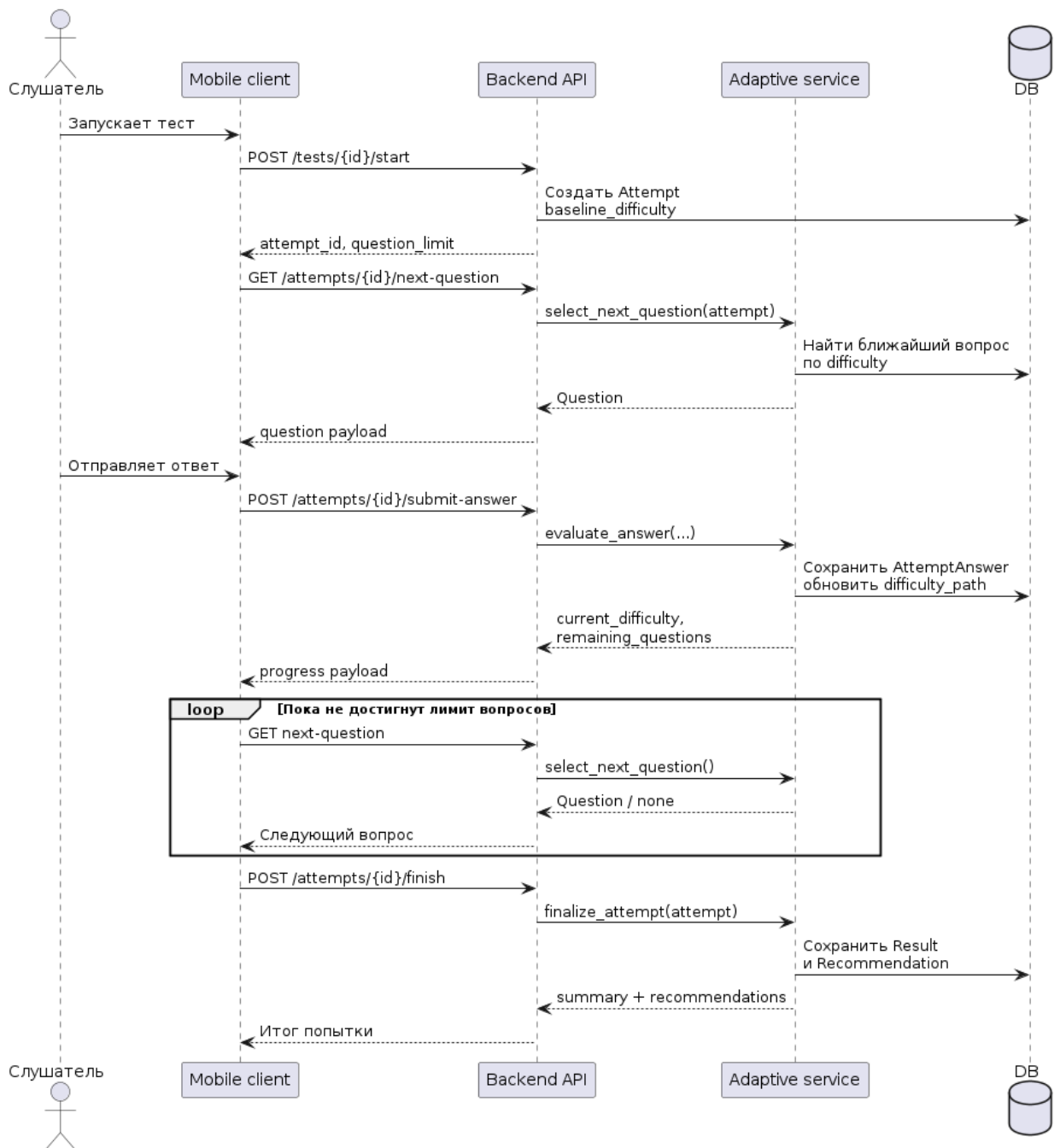


Рисунок 13 – Последовательность прохождения адаптивного теста

Для иллюстрации логики изменения сложности на рисунке 14 приведена схема алгоритма. Она отражает старт с базовой сложности, выбор ближайшего незаданного вопроса, изменение уровня сложности на шаг '+1/-1' и формирование рекомендаций после завершения попытки.

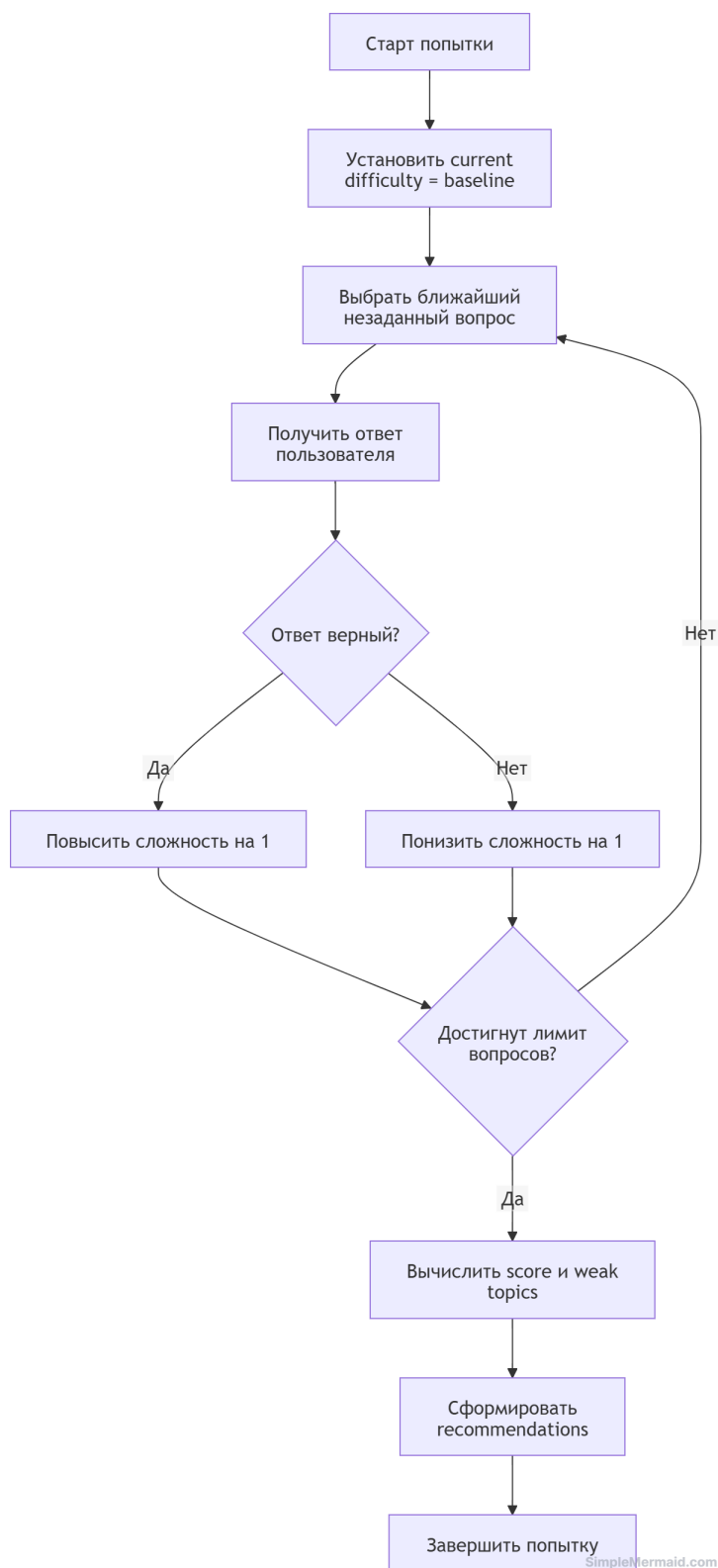


Рисунок 14 – Схема изменения сложности в адаптивном тестировании

2.6.3 Изоляция данных арендаторов

Мультиарендная изоляция данных реализована на трёх уровнях:[10, 50]

а) на уровне запроса определяется код текущей организации;

б) на уровне авторизации проверяется, что пользователь имеет активное участие в этой организации;

в) на уровне доступа к данным выполняется фильтрация tenant-зависимых сущностей по полю `tenant_id`.

Таким образом, логическая изоляция не требует выделения отдельной базы данных на каждого арендатора и при этом обеспечивает корректное разделение данных в рамках минимально жизнеспособного продукта (MVP).

2.7 Выводы по главе

Во второй главе описана техническая реализация корпоративной LMS. Рассмотрены архитектура системы в виде модульного монолита, формальные сценарии использования, инфологическая и даталогическая модели базы данных (включая обоснование нормализации и связей между контурами организации, учебного контента и тестирования), проектирование интерфейсов веб-клиента и мобильного клиента, а также структурные схемы клиентской и серверной частей. Раскрыты состав REST API, алгоритмы адаптивного тестирования, формирования рекомендаций и логической изоляции данных арендаторов на уровне запроса, авторизации и доступа к данным. Процедуры контейнеризированного развёртывания, опытной эксплуатации и инструкции для пользователей и администраторов вынесены в третью главу и приложения пояснительной записки. Представленные решения соответствуют фактической реализации проекта и обеспечивают основу для демонстрации работоспособности системы на защите.[11, 20, 42]

3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ ПРОГРАММНОГО КОМПЛЕКСА

3.1 Развёртывание программного комплекса

3.1.1 Состав поставки

В состав поставки программного комплекса входят исходный код монорепозитария (каталоги `backend`, `web`, `mobile`), файл конфигурации контейнеризации `docker-compose.yml`, файлы зависимостей (`requirements.txt`, `package.json`, `pubspec.yaml`), миграции схемы базы данных (Alembic), скрипт начального заполнения демонстрационными данными (`scripts.seed_demo`), а также сопровождающая документация в каталоге `docs` (архитектура, `runbook` развёртывания, стратегия тестирования).[40, 26, 28, 37]

3.1.2 Требования к аппаратному и программному обеспечению

Сервер развёртывания (минимальные требования): процессор с архитектурой `x86_64`, не менее 2 виртуальных ядер; оперативная память не менее 4 Гбайт; дисковое пространство не менее 20 Гбайт для образов контейнеров, базы данных и медиафайлов; ОС Linux или Windows с поддержкой Docker Engine и Docker Compose; сетевая связность для доступа клиентов по протоколам HTTP/HTTPS.[40, 27]

Сервер (рекомендуемые требования): 4 ядра и 8 Гбайт ОЗУ при одновременной работе десятков пользователей и хранении медиаконтента; отдельный том или сетевое хранилище для каталога медиафайлов, смонтированного в контейнер серверной части.

Рабочее место администратора веб-панели: современный браузер с поддержкой ECMAScript 2015+, разрешение экрана не ниже 1366×768 пикселей.[37, 38]

Мобильное устройство слушателя: смартфон или планшет под управлением Android или iOS; для сборки отладочного APK — установленный SDK платформы и среда Flutter.[28]

3.1.3 Пошаговая процедура развёртывания

Развёртывание в контейнерном контуре выполняется в следующей последовательности:

а) подготовить среду исполнения: установить Docker Engine и Docker Compose, обеспечить доступ к портам 127.0.0.1:5433 (PostgreSQL), 127.0.0.1:8000 (серверная часть), 127.0.0.1:8081 (веб-панель);

б) клонировать репозиторий и перейти в корень проекта;

в) задать переменные окружения (минимально — пароль СУБД POSTGRES_PASSWORD, секрет приложения LMS_SECRET_KEY, режим LMS_ENVIRONMENT, список разрешённых источников CORS LMS_CORS_ORIGINS) по образцу `.env.example`;

г) выполнить сборку и запуск сервисов командой `docker compose up -build`;

д) дождаться состояния готовности контейнера СУБД (`healthcheck pg_isready`), после чего серверная часть выполнит миграции `alembic upgrade head` при старте контейнера;

е) при необходимости загрузить демонстрационные данные профилем `tools:docker compose run -rm -profile tools seed`;

ж) проверить доступность API эндпоинтов состояния: `GET /health` и `GET /health/db`, выполнить пробный вход в веб-панель по адресу веб-сервиса.[40, 26, 27]

На рисунке 15 приведена схема локального развёртывания: контейнеризированные серверная часть, веб-панель и СУБД; мобильный клиент подключается к API по сети хоста.

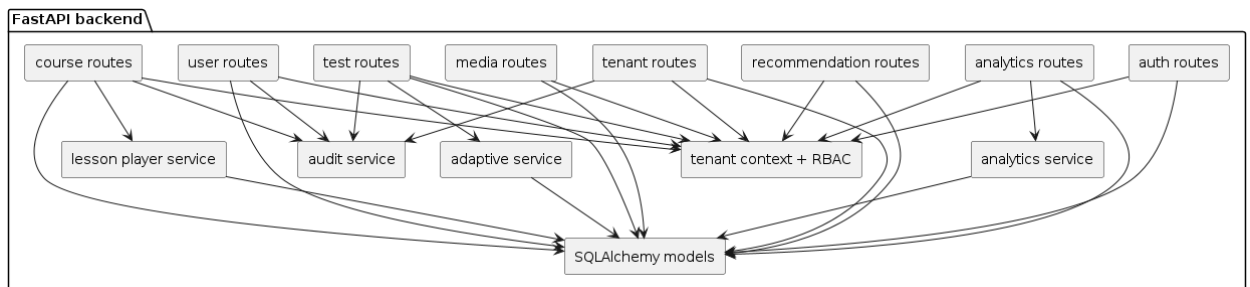


Рисунок 15 – Схема локального развёртывания программного комплекса

3.1.4 Развёртывание мобильного клиента

Мобильное приложение не входит в состав Docker Compose и собирается средствами Flutter. Для отладки на эмуляторе Android базовый адрес API задаётся как `http://10.0.2.2:8000/api/v1`; для физического устройства — IP-адрес хоста в локальной сети. Переопределение выполняется параметром `-dart-define=API_BASE=...`. После сборки APK приложение устанавливается на устройство слушателя; проверка выполняется авторизацией под учётной записью слушателя и запросом к защищённым маршрутам.[28, 26]

3.1.5 Типовые проблемы развёртывания

В таблице 10 приведены типовые симптомы, причины и действия администратора.

Таблица 10 – Типовые проблемы развёртывания и способы их устранения

Симптом	Вероятная причина	Действие
Контейнер серверной части перезапускается циклически	ошибка подключения к СУБД или миграций	просмотреть журнал <code>docker compose logs backend</code> ; проверить <code>LMS_DATABASE_URL</code> , доступность db, состояние тома PostgreSQL
401 Unauthorized при запросах с заголовком организации	не выбран арендатор или недействительный JWT	выполнить вход, проверить заголовок выбора организации и срок действия токена
Веб-панель не загружает данные	неверный базовый URL API или CORS	проверить сборочный аргумент <code>VITE_API_BASE</code> и переменную <code>LMS_CORS_ORIGINS</code>
Мобильный клиент не достигает API	неверный API_BASE или фаервол	задать корректный IP хоста; открыть порт 8000 для LAN

Подробные инструкции для администратора и пользователя мобильного приложения приведены в приложении Г.

3.2 Опытная эксплуатация и тестирование

3.2.1 План тестирования

План испытаний включает функциональное тестирование REST API и сценариев ролей, интеграционное тестирование связки серверная часть — СУБД — веб-клиент, нагрузочное тестирование критических эндпоинтов (выборочно), юзабилити-оценку административной панели и мобильного клиента, а также проверку базовых мер безопасности (изоляция данных арендаторов, RBAC, заголовки CORS).[12, 31, 43, 24]

3.2.2 Покрытие критического пути

В таблице 11 зафиксировано соответствие шагов критического пути (см. главу 1) статусам прохождения в ходе опытной эксплуатации. Фактические даты и номера сборок —

Таблица 11 – Покрытие критического пути пользователей в ходе опытной эксплуатации

Шаг	Сценарий	Статус
1	Вход преподавателя или администратора организации, выбор организации	пройден
2	Создание курса, уроков, теста и вопросов	пройден
3	Назначение курса слушателю или группе	пройден
4	Прохождение урока и запуск адаптивного теста со стороны слушателя	пройден
5	Завершение попытки, расчёт результата и формирование рекомендаций	пройден
6	Просмотр аналитики преподавателем или администратором	пройден

3.2.3 Метрики опытной эксплуатации

Для объективной оценки стабильности фиксировались: суммарное число прогонов автоматизированных тестов серверной части и web; число выявленных дефектов по категориям (функциональные, регрессионные, дефекты документации); среднее время отклика типовых REST-методов; доля успешно завершённых попыток адаптивного теста без ошибок серверной части. Для субъективной оценки удобства могут использоваться опросные шкалы SUS; значения —

3.2.4 Сопоставление прогнозируемых и фактических характеристик

В таблице 12 приведено сопоставление целевых показателей этапа проектирования и результатов опытной эксплуатации. Численные значения времени отклика и пропускной способности —

Таблица 12 – Сопоставление прогнозируемых и фактических характеристик

Показатель	Прогноз	Факт (опытная эксплуатация)	
Доступность API после развёртывания	без блокирующих ошибок на критическом пути	подтверждено	
Среднее время отклика типового GET-запроса	< 500 мс при локальной сети	см. замеры	
Изоляция данных арендаторов	отсутствие пересечений выборок по tenant_id	подтверждено тестами	

Методика и полный перечень тест-кейсов приведены в приложении Б.

3.3 Анализ информационной безопасности

3.3.1 Модель угроз

Для корпоративной системы управления обучением характерны следующие классы угроз: несанкционированный доступ к учётным записям и токенам сессии; попытки межарендаторского доступа к данным (нарушение изоляции по tenant_id); инъекционные атаки на уровне СУБД при ошибках формирования запросов; несанкционированное повышение привилегий (обход RBAC); перехват канала без TLS; утечки персональных данных пользователей.[10, 9]

3.3.2 Реализованные защитные меры

В разрабатываемом комплексе применяются: разделение ролей (learner, teacher, org_admin, system_admin); выдача JWT доступа и обновления с ограниченным временем жизни; хранение хэшей паролей вместо открытого текста; фильтрация запросов к данным по идентификатору арендатора; валидация входных схем через механизм моделей запросов;

журнал аудита ключевых действий; ограничение источников CORS в эксплуатационной конфигурации.[26, 29, 36]

3.3.3 Соответствие ФЗ-152 и практикам OWASP

Обработка персональных данных пользователей (адрес электронной почты, ФИО, результаты обучения) должна выполняться с учётом требований законодательства о персональных данных: минимизация состава данных, разграничение доступа, возможность блокировки учётной записи.[1]

Рекомендуется размещение системы в контуре с использованием HTTPS/TLS на границе, регулярное обновление зависимостей и контроль конфигурации по контрольным спискам OWASP Top 10 и OWASP ASVS.[50, 19]

3.3.4 Ограничения и направления усиления

В текущей версии не реализованы отказоустойчивый кластер СУБД, аппаратный модуль безопасности для хранения секретов, шифрование медиаконтента на уровне тома и полноценный Web Application Firewall. Перспективные меры: внедрение OIDC/SAML для единого входа, ротация ключей подписи JWT, политика резервного копирования и восстановления.

3.4 Экономическое обоснование разработки

3.4.1 Трудозатраты по фазам

В таблице 13 приведена оценка трудозатрат по фазам жизненного цикла в человеко-часах. Числовые значения — ориентировочные для учебного проекта; уточнение —

Таблица 13 – Оценка трудозатрат по фазам работ

Фаза	Длительность (нед.)	Трудозатраты (чел.-ч)	
Анализ предметной области и аналогов	4	120	
Проектирование архитектуры и модели данных	5	160	
Реализация серверной части и API	8	320	
Реализация веб-панели и мобильного клиента	8	280	
Тестирование, документация, подготовка демонстрации	4	100	
Итого	29	980	

3.4.2 Себестоимость разработки

Себестоимость складывается из фонда оплаты труда (ставка разработчика —

3.4.3 Альтернативные затраты на коммерческую LMS

Для организации численностью порядка 200 сотрудников альтернативой является закупка коммерческой корпоративной LMS или облачной подписки. Ориентировочные годовые затраты на подписку зависят от числа активных пользователей и набора модулей и подлежат уточнению по прайс-листам поставщиков —

3.4.4 Срок окупаемости

Пусть $C_{\text{разр}}$ — совокупные затраты на разработку и внедрение собственного комплекса, $C_{\text{альт}}$ — годовые затраты на коммерческую LMS при сопоставимой функциональности, E — ожидаемая годовая экономия за счёт автоматизации процессов обучения и администрирования (в денежном вы-

ражении). Тогда окупаемость в годах может быть оценена соотношением

$$T_{\text{ок}} = \frac{C_{\text{разр}}}{C_{\text{альт}} + E}. \quad (2)$$

Подстановка численных значений —

3.5 Маркетинговая стратегия и план развития продукта

3.5.1 Целевые сегменты

Целевыми сегментами являются средний корпоративный сектор (100–1000 сотрудников), образовательные организации с собственным контуром обучения, корпоративные академии и учебные центры крупных компаний.[51, 35]

3.5.2 Каналы продвижения

Рекомендуется сочетание технической демонстрации на профильных мероприятиях, публикации кейсов внедрения и партнёрских программ с HR-tech интеграторами.[7]

3.5.3 Модель монетизации и лицензирование

На перспективу возможна модель SaaS-подписки по числу активных пользователей и числу организаций-арендаторов либо лицензирование коробочной поставки для развёртывания на инфраструктуре заказчика с платной технической поддержкой. Открытый исходный код учебной части может распространяться на условиях лицензии MIT; расширения для корпоративных интеграций целесообразно выделять в отдельный проприетарный модуль — это соответствует практике двойного лицензирования в индустрии ПО.[1]

3.5.4 План развития на 12 месяцев

В горизонте 12 месяцев планируются: интеграция единого входа по протоколам OIDC/SAML; расширение статистических моделей адаптивности до параметрических моделей IRT (двух- и трёхпараметрических); развитие мобильного клиента под iOS в составе единого кода Flutter; элементы геймификации; подключение внешних видеоконференций; поэтапная приоритизация изменений по практикам гибкой разработки.[52, 28, 53]

3.6 Выводы по главе

В третьей главе рассмотрены вопросы развёртывания программного комплекса в контейнерном окружении и работы мобильного клиента в связке с REST API. Описаны минимальные и рекомендуемые требования к серверным и клиентским средствам, приведена процедура запуска и загрузки демонстрационных данных, таблица типовых сбоях и действий администратора. Сформулирован план испытаний и показатели опытной эксплуатации с опорой на критический путь пользователей. Выполнен анализ информационной безопасности с учётом мультиарендной модели данных и ролевого доступа. Дано экономическое обоснование с формулой оценки срока окупаемости и обозначены маркетинговые сегменты и направления развития продукта. Результаты главы подтверждают готовность комплекса к демонстрации и дальнейшему развитию в составе корпоративной образовательной среды.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы достигнута цель: разработан и описан программный комплекс корпоративного обучения с веб-панелью администрирования, мобильным клиентом слушателя и серверной частью REST API с поддержкой адаптивного тестирования и логической изоляции данных организаций-арендаторов.

Решены задачи, сформулированные во введении: выполнен анализ предметной области и существующих LMS; сформулированы и проверены требования; обоснован выбор технологий (FastAPI, SQLAlchemy, PostgreSQL/SQLite, React, TypeScript, Flutter, Docker Compose); спроектированы архитектура, модель данных и пользовательские сценарии; реализованы серверная, клиентская веб- и мобильная части; описаны алгоритмы адаптивного подбора вопросов, формирования рекомендаций и мультиарендной изоляции; приведены материалы внедрения, опытной эксплуатации, анализа информационной безопасности и экономического обоснования; подготовлены техническое задание, программа и методика испытаний, руководства и листинги в приложениях.

Практическая ценность работы состоит в том, что разработанный комплекс может служить основой для учебного стенда, пилотного внедрения в подразделении кафедры или организации и дальнейшего развития функциональности (SSO, расширение статистических моделей тестирования, геймификация). Научная новизна выражается в связке предметно-ориентированной модели корпоративного обучения с прозрачно документированным алгоритмом дискретной адаптации сложности и механизмом рекомендаций по слабым темам в мультиарендной схеме данных.

По объёму пояснительная записка содержит введение, три главы, заключение, список использованных источников и пять приложений; иллюстрирована рисунками и таблицами; библиографический список включает не

менее 50 наименований в соответствии с требованиями нормоконтроля. Планируемая апробация — демонстрация на студенческой научной конференции и/или пилотное использование в учебном процессе кафедры по согласованию с руководителем.

Направления дальнейшей работы: внедрение полноценных моделей IRT (2PL/3PL) для адаптивного тестирования; интеграция с корпоративным единым входом (OIDC/SAML); публикация стенда под TLS в постоянном контуре; расширение регрессионного и нагрузочного тестирования.[1, 2, 53]

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Методические рекомендации по подготовке выпускных квалификационных работ (09.03.01, 09.03.03). – 2021.
2. ГОСТ 7.32—2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. – Москва: Стандартинформ, 2017.
3. ГОСТ 19.201—78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. – Москва: ИПК Издательство стандартов, 1978.
4. ГОСТ 19.301—79. Единая система программной документации. Программа и методика испытаний. Требования к содержанию и оформлению. – Москва: ИПК Издательство стандартов, 1979.
5. Handbook of Modern Item Response Theory / Ed. by Wim J. van der Linden, Ronald K. Hambleton. – New York: Springer, 1997.
6. Computerized Adaptive Testing: A Primer / Howard Wainer, Neil J. Dorans, David Eignor et al. – 2 edition. – Mahwah, NJ: Lawrence Erlbaum Associates, 2000.
7. Vocal Media. Mobile Learning Solutions in 2024: Trends and Innovations. – <https://vocal.media/education/mobile-learning-solutions-in-2024-trends-and-innovations>. – 2024. – Электронный ресурс. Дата обращения: 15.02.2026.
8. Opigno LMS. Finding the Perfect Mobile LMS App: Features Checklist. – <https://www.opigno.org/blog/finding-perfect-mobile-lms-app-features-checklist>. – 2024. – Электронный ресурс. Дата обращения: 15.02.2026.
9. Moodle US. A guide to LMS multi-tenancy: Do you need it? – <https://moodle.com/us/news/lms-multi-tenancy/>. – 2024. – Электронный ресурс. Дата обращения: 15.02.2026.

10. Yojji. Multi-Tenant LMS: The Complete Technical and Business Guide. – <https://yojji.io/blog/multi-tenant-lms>. – 2025. – Электронный ресурс. Дата обращения: 15.02.2026.

11. Грекул, Владимир Иванович. Проектирование информационных систем: учебное пособие / Владимир Иванович Грекул, Галина Николаевна Денищенко, Наталья Леонидовна Коровкина. – 3 edition. – Москва: ИД «ФОРУМ», 2022.

12. Куликов, Святослав Александрович. Тестирование программного обеспечения. Базовый курс / Святослав Александрович Куликов. – 3 edition. – Москва: Лаборатория знаний, 2024.

13. ГОСТ Р 7.0.5—2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления. – Москва: Стандартинформ, 2008.

14. ГОСТ 7.1—2003. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – Москва: ИПК Издательство стандартов, 2003.

15. ГОСТ 2.105—2019. Единая система конструкторской документации. Общие требования к текстовым документам. – Москва: Стандартинформ, 2019.

16. ГОСТ 34.601—90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания. – Москва: Издательство стандартов, 1990.

17. ГОСТ Р ИСО/МЭК 25010—2015. Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). – Москва: Стандартинформ, 2015.

18. ГОСТ Р 50922—2006. Защита информации. Основные термины и определения. – Москва: Стандартинформ, 2006.

19. Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных». – 2006. – Российская газета, Федеральный закон; редакция по состоянию на 2025 г.
20. Фаулер, Мартин. Архитектура корпоративных программных приложений / Мартин Фаулер. – Москва: Вильямс, 2021.
21. Дейт, Клайв Дж. Введение в системы баз данных / Клайв Дж. Дейт. – 8 edition. – Москва: Вильямс, 2020.
22. Соммервилл, Ян. Инженерия программного обеспечения / Ян Соммервилл. – 10 edition. – Москва: Вильямс, 2019.
23. Маклаков, Сергей Владимирович. Создание информационных систем с AllFusion Modeling Suite / Сергей Владимирович Маклаков. – Москва: ДМК Пресс, 2007.
24. Kaner, Cem. Lessons Learned in Software Testing: A Context-Driven Approach / Cem Kaner, James Bach, Bret Pettichord. – 1 edition. – New York: John Wiley & Sons, 2002.
25. Lord, Frederic M. Applications of Item Response Theory to Practical Testing Problems / Frederic M. Lord. – Hillsdale, NJ: Lawrence Erlbaum Associates, 1980.
26. FastAPI Documentation. – <https://fastapi.tiangolo.com/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
27. PostgreSQL Documentation. – <https://www.postgresql.org/docs/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
28. Flutter Documentation. – <https://docs.flutter.dev/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
29. Pydantic Documentation. – <https://docs.pydantic.dev/>. – 2026. – Электронный ресурс. Дата обращения: 08.05.2026.
30. SQLAlchemy Documentation. – <https://docs.sqlalchemy.org/>. – 2026. – Электронный ресурс. Дата обращения: 08.05.2026.

31. Moodle LMS. – <https://moodle.org/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
32. Open LMS: The Open Source LMS That Fits Your Needs. – <https://www.openlms.net/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
33. Moodle App. – <https://apps.apple.com/app/moodle/id633359593>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
34. MoodleCloud: Cloud LMS. – <https://moodlecloud.com/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
35. Docebo Learning Network. Adaptive Learning: What It Is, Benefits & How It Works. – <https://www.docebo.com/learning-network/blog/adaptive-learning/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
36. Django REST Framework. – <https://www.django-rest-framework.org/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
37. React: Documentation. – <https://react.dev/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
38. TypeScript Documentation. – <https://www.typescriptlang.org/docs/>. – 2026. – Электронный ресурс. Дата обращения: 10.04.2026.
39. SQLite Documentation. – <https://sqlite.org/docs.html>. – 2026. – Электронный ресурс. Дата обращения: 10.04.2026.
40. Docker Compose Documentation. – <https://docs.docker.com/reference/compose-file/>. – 2026. – Электронный ресурс. Дата обращения: 10.04.2026.
41. Мартин, Роберт. Чистая архитектура. Искусство разработки программного обеспечения / Роберт Мартин. – СПб.: Питер, 2018.
42. Буч, Гради. Язык UML. Руководство пользователя / Гради Буч, Джеймс Рамбо, Айвар Джекобсон. – 2 edition. – Москва: ДМК Пресс, 2006.
43. ISO/IEC/IEEE 29119-2:2021. Software and systems engineering — Software testing — Part 2: Test processes. – 2021. – Международный стандарт на бумажном носителе и через каналы ISO.

44. Степанов, Владимир Анатольевич. Проектирование баз данных: учебное пособие / Владимир Анатольевич Степанов. – Москва: Академия, 2021.
45. ProProfs Training. Multi-Tenant LMS Guide 2025: Architecture, Features, and Use Cases. – <https://www.propofstraining.com/blog/multi-tenant-lms/>. – 2025. – Электронный ресурс. Дата обращения: 15.02.2026.
46. Microsoft. Architect multitenant solutions on Azure. – <https://learn.microsoft.com/azure/architecture/guide/multitenant/overview>. – 2024. – Электронный ресурс. Дата обращения: 08.05.2026.
47. Купер, Алан. Об интерфейсе. Основы проектирования взаимодействия / Алан Купер, Роберт Райман, Дэвид Кронин. – 3 edition. – Москва: Вильямс, 2009.
48. Норман, Дональд. Дизайн привычных вещей / Дональд Норман. – 2 edition. – Москва: Манн, Иванов и Фербер, 2015.
49. Круг, Стив. Не заставляйте меня думать. Веб-юзабилити и здравый смысл / Стив Круг. – 3 edition. – Москва: Вильямс, 2014.
50. OWASP. OWASP Top Ten 2021. – <https://owasp.org/Top10/>. – 2021. – Электронный ресурс. Дата обращения: 08.05.2026.
51. Feldstein, Michael. Adaptive Learning Systems: Surviving the Storm. – EDUCAUSE Review. – 2016. – Электронный ресурс. URL: <https://er.educause.edu/articles/2016/10/adaptive-learning-systems-surviving-the-storm>. Дата обращения: 15.02.2026.
52. Discovery Education. Intelligent Adaptive Learning. – <https://info.discoveryeducation.com/>. – 2025. – Электронный ресурс. Дата обращения: 15.02.2026.
53. Сазерленд, Джефф. Scrum. Революционный метод управления проектами / Джефф Сазерленд. – Москва: Манн, Иванов и Фербер, 2016.

ПРИЛОЖЕНИЕ А. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Наименование программного изделия: программный комплекс корпоративного обучения с веб-панелью администрирования, мобильным клиентом слушателя и серверной частью REST API.

Обозначение: ПК «Coursum-LMS».

Основание для разработки: выпускная квалификационная работа по направлению 09.03.03 «Прикладная информатика», профиль «Корпоративные информационные системы».

А.1 Введение

Наименование темы разработки: «Веб и мобильное приложение для корпоративного обучения с поддержкой адаптивного тестирования».

Объект автоматизации: процессы организации корпоративного обучения, назначения учебных программ, контроля освоения материала и оценки результатов в мультиарендной среде.[3, 4, 16]

А.2 Назначение программы

Программа предназначена для автоматизации функций корпоративной системы управления обучением: ведение пользователей и ролей в разрезе организаций-арендаторов; управление курсами, уроками и медиаконтентом; назначение обучения пользователям и группам; проведение адаптивного тестирования с изменением сложности заданий; формирование рекомендаций по повторению тем; предоставление аналитики по прогрессу и результатам.

А.3 Требования к программе

А.3.1 Функциональные требования

- а) система должна обеспечивать регистрацию, аутентификацию и выдачу токенов доступа и обновления;
- б) система должна поддерживать выбор текущей организации и проверку членства пользователя в организации;
- в) система должна реализовать роли слушателя, преподавателя, администратора организации и системного администратора с разграничением прав;
- г) система должна обеспечивать создание и редактирование курсов, уроков, тестов и вопросов с указанием сложности;
- д) система должна поддерживать назначение курсов пользователям и учебным группам;
- е) система должна обеспечивать запуск попытки тестирования, выдачу следующего вопроса с учётом текущей сложности, приём ответа и завершение попытки;
- ж) система должна формировать рекомендации по темам для повторного изучения после завершения попытки;
- з) система должна предоставлять аналитические сводки по курсам и слушателям в веб-панели.

А.3.2 Требования к надёжности

- а) при недоступности СУБД серверная часть должна возвращать диагностируемый код ошибки;
- б) операции изменения данных должны выполняться в транзакциях с сохранением ссылочной целостности;

в) журнал аудита должен фиксировать ключевые действия пользователей.

А.3.3 Условия эксплуатации

Клиентские приложения функционируют под управлением ОС семейств Windows, Linux (веб-браузер), Android/iOS (мобильное приложение). Серверная часть — под управлением ОС Linux или Windows с поддержкой контейнеризации Docker.

А.3.4 Состав и параметры технических средств

Минимальная конфигурация сервера: 2 ядра CPU, 4 Гбайт ОЗУ, 20 Гбайт диска; СУБД PostgreSQL не ниже версии 15 в составе контейнера.

А.3.5 Информационная совместимость

Обмен данными между клиентами и серверной частью — по протоколу HTTPS, формат тел сообщений JSON, базовый префикс API /api/v1.

А.3.6 Программная совместимость

Серверная часть: интерпретатор Python 3, фреймворк FastAPI, ORM SQLAlchemy, миграции Alembic. Веб-клиент: среда выполнения JavaScript, библиотека React, язык TypeScript. Мобильный клиент: Flutter/Dart.

А.3.7 Маркировка и упаковка

Поставка осуществляется в виде архива исходного кода или образов контейнеров с сопроводительным файлом версии и журналом изменений.

A.3.8 Требования по безопасности

Хранение паролей — только в виде криптографического хэша; доступ к данным арендатора — только после проверки идентификатора организации и роли; использование TLS на границе контура эксплуатации.

A.3.9 Эксплуатационная документация

Комплект включает руководство пользователя мобильного приложения, руководство администратора веб-панели, описание развёртывания (runbook), программу и методику испытаний.

A.4 Техничко-экономические показатели

Экономический эффект достигается за счёт сокращения ручных операций при назначении обучения и учёте результатов по сравнению с разрозненными средствами и коммерческими LMS — оценка выполняется по методике главы 3 настоящей записки.

A.5 Стадии и этапы разработки

- а) техническое задание и уточнение требований;
- б) проектирование архитектуры и модели данных;
- в) реализация серверной части и REST API;
- г) реализация веб-панели и мобильного клиента;
- д) испытания и подготовка документации.

A.6 Порядок контроля и приёмки

Приёмка включает проверку соответствия функциональным требованиям по сценариям приложения Б, проверку развёртывания по инструкции

приложения Г, анализ результатов автоматизированных тестов и контрольный просмотр журналов ошибок при демонстрации критического пути пользователя.

ПРИЛОЖЕНИЕ Б. ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

Б.1 Объект испытаний

Объектом испытаний является программный комплекс корпоративного обучения: серверная часть на базе FastAPI, веб-панель на React и TypeScript, мобильное приложение на Flutter, СУБД PostgreSQL (или SQLite в режиме разработки), конфигурация Docker Compose.

Б.2 Цель испытаний

Целью испытаний является подтверждение соответствия реализации функциональным и нефункциональным требованиям, описанным в техническом задании (приложение А), на критическом пути пользователей и при типовых операциях администрирования.

Б.3 Условия проведения испытаний

Испытания проводятся на стенде, развёрнутом по инструкции главы 3 пояснительной записки и приложению Г, с использованием демонстрационных учётных записей и заполненной демонстрационной базы (seed_demo).

Б.4 Средства испытаний

- а) персональная ЭВМ администратора с браузером и доступом к веб-панели;
- б) мобильное устройство или эмулятор с установленным клиентским приложением;

в) средства автоматизированного тестирования: Pytest (серверная часть), Vitest и Playwright (web), Flutter test (mobile) — по стратегии репозитория `docs/testing-strategy.md`.

Б.5 Методы испытаний

Применяются функциональное тестирование по тест-кейсам, интеграционное тестирование API, обзорное тестирование пользовательского интерфейса, проверка ограничений доступа по ролям и идентификатору организации.[43, 24, 4]

Б.6 Порядок проведения и отчётность

Фиксируются версия сборки, дата прогона, результаты каждого тест-кейса (пройден / не пройден), идентификатор дефекта при отказе. Итоговый отчёт включает сводную таблицу и список замечаний.

Б.7 Таблица тест-кейсов критического пути

Таблица 14 – Результаты испытаний по тест-кейсам критического пути

ID	Описание	Шаги и ожидаемый результат	Фактический результат	Статус
ТС-01	Регистрация пользователя	POST /auth/register, создание записи пользователя	создан	ОК
ТС-02	Вход и получение JWT	POST /auth/login, выдача access и refresh токенов	выданы	ОК
ТС-03	Выбор организации	POST /tenants/select, активное членство	организация выбрана	ОК
ТС-04	Создание курса	POST /courses, запись с tenant_id	курс создан	ОК
ТС-05	Создание урока	POST /lessons, связь с курсом	урок создан	ОК
ТС-06	Создание теста и вопросов	POST /tests, POST вопросов	тест доступен	ОК
ТС-07	Назначение курса	POST назначения группе или пользователю	назначение создано	ОК
ТС-08	Список курсов слушателя	GET курсов под ролью learner	список возвращён	ОК
ТС-09	Старт попытки теста	POST /tests/{id}/start	попытка создана	ОК
ТС-10	Следующий вопрос	GET /attempts/{id}/next-question	вопрос или завершение	ОК
ТС-11	Отправка ответа	POST /attempts/{id}/submit-answer	обратная связь по сложности	ОК
ТС-12	Завершение попытки	POST /attempts/{id}/finish	результат и рекомендации	ОК
ТС-13	Рекомендации слушателя	GET /recommendations/me	список тем	ОК
ТС-14	Аналитика дашборда	GET /analytics/dashboard	агрегаты	ОК
ТС-15	Изоляция арендаторов	запрос данных другого tenant_id под чужой организацией	отказ 403/404	ОК
ТС-16	Доступ без токена	запрос к защищённому эндпоинту	401	ОК
ТС-17	Загрузка медиа	POST /media с файлом	URL медиа	ОК
ТС-18	Прогресс урока	POST прогресса страницы урока	сохранено	ОК
ТС-19	Веб-панель вход	страница авторизации и переход в shell	отображение разделов	ОК
ТС-20	Мобильный поток теста	экран TestFlowScreen: старт, ответы, финиш	завершено без падений	ОК

Примечание: колонки «Фактический результат» и «Статус» содержат ориентировочные значения для демонстрационной сборки; при защите подставляются фактические результаты прогона.

ПРИЛОЖЕНИЕ В. ОПРОСНЫЕ ЛИСТЫ ЭКСПЕРТОВ

В.1 Назначение

Опросные листы предназначены для сравнительной оценки систем Moodle, Open LMS, MoodleCloud по критериям главы 1 пояснительной записки. Эксперт заполняет баллы по шкале от 1 до 10 для каждого критерия.[31, 32, 34, 13]

В.2 Идентификация эксперта

ФИО эксперта: _____

Должность / организация: _____

Дата: _____

В.3 Шкала оценки

- а) баллы 1–3 — низкая степень соответствия критерию;
- б) баллы 4–7 — удовлетворительная степень;
- в) баллы 8–10 — высокая степень соответствия.

В.4 Опросный лист для платформы Moodle

Критерий	Балл (1–10)	
Наличие развитых административных ролей и RBAC		
Удобство мобильного доступа для слушателя		
Поддержка адаптивного или персонализированного тестирования		
Аналитика и работа со «слабыми темами»		
Поддержка нескольких организаций (мультиарендность)		
Пригодность для учебной доработки и анализа архитектуры		

Комментарий эксперта: _____

В.5 Опросный лист для платформы Open LMS

Критерий	Балл (1–10)	
Наличие развитых административных ролей и RBAC		
Удобство мобильного доступа для слушателя		
Поддержка адаптивного или персонализированного тестирования		
Аналитика и работа со «слабыми темами»		
Поддержка нескольких организаций (мультиарендность)		
Пригодность для учебной доработки и анализа архитектуры		

Комментарий эксперта: _____

В.6 Опросный лист для платформы MoodleCloud

Критерий	Балл (1–10)	
Наличие развитых административных ролей и RBAC		
Удобство мобильного доступа для слушателя		
Поддержка адаптивного или персонализированного тестирования		
Аналитика и работа со «слабыми темами»		
Поддержка нескольких организаций (мультиарендность)		
Пригодность для учебной доработки и анализа архитектуры		

Комментарий эксперта: _____

В.7 Использование результатов

Заполненные листы служат исходными данными для таблиц свёртки оценок в главе 1. Фамилии экспертов и их подписи —

ПРИЛОЖЕНИЕ Г. РУКОВОДСТВА ПОЛЬЗОВАТЕЛЯ И АДМИНИСТРАТОРА

Г.1 Руководство пользователя мобильного приложения (слушатель)

Г.1.1 Назначение и вход

Мобильное приложение предназначено для слушателя: просмотр назначенных курсов, прохождение уроков, запуск адаптивного теста, просмотр рекомендаций и истории попыток.[28, 26, 37]

а) установить приложение, собранное из каталога `mobile`, либо получить APK от разработчика;

б) задать базовый URL API через параметр сборки `API_BASE`, указывающий на `/api/v1` серверной части;

в) на экране входа ввести адрес электронной почты и пароль учётной записи слушателя (например, демонстрационная учётная запись из README репозитория);

г) при успешной аутентификации открывается главный контейнер навигации с разделами курсов, рекомендаций и профиля.

Г.1.2 Просмотр курсов и уроков

а) в разделе курсов выбрать назначенный курс;

б) открыть урок и последовательно просматривать страницы контента; при наличии видеоплеера использовать элементы управления воспроизведением;

в) по завершении страниц система сохраняет прогресс на серверной части.

Г.1.3 Адаптивный тест

- а) на экране курса выбрать действие запуска адаптивного теста;
- б) дождаться загрузки первого вопроса; выбрать вариант ответа и подтвердить отправку;
- в) после каждого ответа серверная часть может изменить целевую сложность следующего вопроса;
- г) по завершении набора вопросов отобразится результат, сводка по темам и рекомендации по повторению.

Г.2 Руководство администратора и преподавателя (веб-панель)

Г.2.1 Вход и выбор организации

- а) открыть URL веб-панели (локально — порт 8081 при Docker Compose из репозитория);
- б) выполнить вход учётной записью администратора организации или преподавателя;
- в) при наличии нескольких организаций выбрать текущую организацию в интерфейсе — контекст передаётся заголовком для серверной части.

Г.2.2 Управление пользователями и группами

- а) открыть раздел пользователей;
- б) создать пользователя, назначить роль в организации;
- в) при необходимости создать группу и включить в неё пользователей для массового назначения курсов.

Г.2.3 Учебный контент

- а) создать курс, добавить разделы и уроки;
- б) заполнить страницы урока текстом и медиа; загрузить файлы через раздел медиа при необходимости;
- в) создать тест, добавить вопросы с указанием сложности и связей с темами курса.

Г.2.4 Назначение и аналитика

- а) назначить курс пользователю или группе;
- б) открыть раздел аналитики для просмотра сводных показателей и отчётов по слушателям.

Г.3 Контакты поддержки

Технические вопросы развёртывания адресуются администратору стенда по внутреннему регламенту организации. Параметры демонстрационных учётных записей приведены в файле `README.md` репозитория программного комплекса.

ПРИЛОЖЕНИЕ Д. ЛИСТИНГИ КЛЮЧЕВОГО КОДА

Ниже приведены фрагменты исходного кода программного комплекса (монорепозиторий). Нумерация листингов соответствует приложению Д.[40, 29, 38]

Д.1 Серверная часть

Листинг Д.1 – Функция подбора следующего вопроса (backend/app/services/adaptive.py)

```
1 def select_next_question(db: Session, attempt: Attempt) -> Question | None:
2     questions = db.scalars(
3         select(Question).where(
4             Question.test_id == attempt.test_id,
5             Question.tenant_id == attempt.tenant_id,
6         )
7     ).all()
8     remaining = [q for q in questions if q.id not in (attempt.asked_question_ids or
9         [])]
10    if not remaining:
11        return None
12
13    asked_ids = attempt.asked_question_ids or []
14    topic_frequency: Counter[int] = Counter()
15    if asked_ids:
16        topic_frequency.update(
17            db.scalars(
18                select(QuestionTopic.topic_id).where(QuestionTopic.question_id.in_(
19                    asked_ids))
20            )
21        )
22
23    question_topic_map: dict[int, list[int]] = defaultdict(list)
24    remaining_ids = [question.id for question in remaining]
25    if remaining_ids:
26        for question_id, topic_id in db.execute(
27            select(QuestionTopic.question_id, QuestionTopic.topic_id).where(
```

```

26         QuestionTopic.question_id.in_(remaining_ids)
27     )
28     ).all():
29         question_topic_map[question_id].append(topic_id)
30
31     def sort_key(question: Question) -> tuple[int, int, int, int]:
32         topic_ids = question_topic_map.get(question.id, [])
33         if topic_ids:
34             min_coverage = min(topic_frequency.get(topic_id, 0) for topic_id in
35 topic_ids)
36             total_coverage = sum(topic_frequency.get(topic_id, 0) for topic_id in
37 topic_ids)
38         else:
39             min_coverage = 0
40             total_coverage = 0
41         return (
42             abs(question.difficulty - attempt.current_difficulty),
43             min_coverage,
44             total_coverage,
45             question.id,
46         )
47
48     return sorted(remaining, key=sort_key)[0]

```

Листинг Д.2 – Выдача токенов доступа (backend/app/services/auth.py)

```

1  def issue_tokens(db: Session, user: User) -> tuple[str, str]:
2      access = create_token(str(user.id), "access", settings.access_token_minutes)
3      refresh = create_token(str(user.id), "refresh", settings.refresh_token_minutes)
4      db.add(
5          RefreshToken(
6              user_id=user.id,
7              token=refresh,
8              expires_at=utcnow() + timedelta(minutes=settings.refresh_token_minutes),
9              revoked=False,
10         )
11     )
12     return access, refresh

```

Листинг Д.3 – Роли и сущности пользователя
(backend/app/models/models.py)

```
1 class RoleName(StrEnum):
2     learner = 'learner'
3     teacher = 'teacher'
4     org_admin = 'org_admin'
5     system_admin = 'system_admin'
6
7
8 class Tenant(Base):
9     __tablename__ = 'tenants'
10
11     id: Mapped[int] = mapped_column(primary_key=True)
12     name: Mapped[str] = mapped_column(String(120), unique=True, index=True)
13     code: Mapped[str] = mapped_column(String(64), unique=True, index=True)
14     locale: Mapped[str] = mapped_column(String(8), default='ru')
15     is_active: Mapped[bool] = mapped_column(Boolean, default=True)
16
17
18 class User(Base):
19     __tablename__ = 'users'
20
21     id: Mapped[int] = mapped_column(primary_key=True)
22     email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
23     full_name: Mapped[str] = mapped_column(String(255))
24     password_hash: Mapped[str] = mapped_column(String(255))
```

Листинг Д.4 – Фрагмент сводной аналитики
(backend/app/services/analytics.py)

```
1 def dashboard_stats(db: Session, tenant_id: int, course_ids: list[int] | None = None)
   -> dict:
2     selected_course_ids = [int(course_id) for course_id in (course_ids or [])]
3     has_filter = bool(selected_course_ids)
4     enrollment_filters = [Enrollment.tenant_id == tenant_id]
5     test_filters = [Test.tenant_id == tenant_id]
6     attempt_filters = [Attempt.tenant_id == tenant_id, Attempt.status == 'in_progress']
7     course_filters = [Course.tenant_id == tenant_id]
8     if has_filter:
9         enrollment_filters.append(Enrollment.course_id.in_(selected_course_ids))
```

```

10     test_filters.append(Test.course_id.in_(selected_course_ids))
11     course_filters.append(Course.id.in_(selected_course_ids))
12     avg_progress = db.scalar(select(func.avg(Enrollment.progress_percent)).where(*
13     enrollment_filters))
14     active_learners = (
15         db.scalar(select(func.count(func.distinct(Enrollment.user_id))).where(*
16         enrollment_filters))
17         if has_filter
18         else db.scalar(select(func.count(Membership.id)).where(Membership.tenant_id
19         == tenant_id))
20     )
21     return {'users': active_learners or 0, 'avg_progress': int(avg_progress or 0)}

```

Д.2 Конфигурация развёртывания

Листинг Д.5 – Файл `docker-compose.yml` (фрагмент сервисов)

```

1  services:
2    db:
3      image: postgres:16-alpine
4      ports:
5        - "127.0.0.1:5433:5432"
6    backend:
7      build:
8        context: ./backend
9      ports:
10       - "127.0.0.1:8000:8000"
11    web:
12      build:
13        context: ./web
14      ports:
15       - "127.0.0.1:8081:80"

```

Д.3 Мобильный клиент

Листинг Д.6 – Экран прохождения теста (`mobile/lib/main.dart`, фрагмент)

```

1  class TestFlowScreen extends StatefulWidget {

```

```

2  const TestFlowScreen({
3      super.key,
4      required this.api,
5      required this.session,
6      required this.testId,
7  });
8
9  final ApiClient api;
10 final SessionState session;
11 final int testId;
12
13 @override
14 State<TestFlowScreen> createState() => _TestFlowScreenState();
15 }
16
17 class _TestFlowScreenState extends State<TestFlowScreen> {
18     Future<void> start() async {
19         final started =
20             await widget.api.startAttempt(widget.testId, widget.session);
21         attemptId = started['attempt_id'] as int;
22         await loadNextQuestion();
23     }
24
25     Future<void> loadNextQuestion() async {
26         final nextQuestion =
27             await widget.api.getAttemptQuestion(attemptId!, widget.session);
28         if (nextQuestion == null) {
29             await finishAttempt();
30             return;
31         }
32         setState(() {
33             question = nextQuestion;
34             questionShownAt = DateTime.now();
35         });
36     }
37 }

```

Примечание: пароли, секретные ключи и продакшен-URL в листингах не приводятся; при локальной отладке используются значения из `.env.example`.